

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования
«Пермский государственный национальный исследовательский университет»

Физико-математический институт

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
на тему
ИНТЕЛЛЕКТУАЛЬНЫЙ СВЕТОФОР. АЛГОРИТМ С
ИСПОЛЬЗОВАНИЕМ КОМПЬЮТЕРНОГО ЗРЕНИЯ ДЛЯ
ПЕРЕКЛЮЧЕНИЯ СИГНАЛОВ СВЕТОФОРА

Направление: 01.04.02 Прикладная математика и информатика

Выполнил:

магистр 2 курса

_____ **Ю. В. Санников**

Научный руководитель:

Директор института, доктор физико-
математических наук

_____ **М.А. Барулина**

Пермь 2026

СОДЕРЖАНИЕ

ОСНОВНЫЕ ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ	4
ВВЕДЕНИЕ.....	5
1. АНАЛИЗ СУЩЕСТВУЮЩИХ РЕШЕНИЙ ДЛЯ АДАПТИВНОГО УПРАВЛЕНИЯ ТРАФИКОМ.....	8
1.1. Адаптивный светофор.....	8
1.2. Использование ML для детекции автомобилей в России	9
1.2.1. Видеодетектор автотранспортных средств ИНФОПРО и Видеодетектор транспорта РТК	9
1.2.2. НПО ИТС: Паук Трафик.....	10
1.3. Использование светофоров, управляемых ИИ, в странах Европы	11
2. АНАЛИЗ АРХИТЕКТУРЫ.....	12
2.1. Выбор архитектуры Baseline	12
2.2. Ultralytics YOLO	13
2.3. CatBoostClassifier	15
3. ПРОГРАММА ДЛЯ АНАЛИЗА ДОРОЖНОЙ СИТУАЦИИ С ИСПОЛЬЗОВАНИЕМ КОМПЬЮТЕРНОГО ЗРЕНИЯ	16
3.1. Поиск и разметка датасета.....	16
3.2. Результаты дообучения модели сегментации.....	18
3.3. Реализация класса, для сегментации видеоряда и получения информации о трафике	19
3.3.1. API-методы для работы с видео.....	20
3.3.2. Методы для сегментации видео	20
3.3.3. Алгоритм для подсчета объектов на светофоре.....	21
3.3.4. Выбор алгоритма для отслеживания объектов	24
3.3.5. Результаты работы класса	25
4. МОДЕЛЬ ДЛЯ ПРЕДСКАЗАНИЯ СИГНАЛА СВЕТОФОРА	27
4.1. Создание датасета для обучения.....	27
4.2. Обучение модели и результаты метрик	28
5. СИМУЛЯЦИЯ ДОРОЖНОГО ДВИЖЕНИЯ С ИСПОЛЬЗОВАНИЕМ SUMO	31
5.1. Методы инициализации дорожной сетки и трафика машин	31

5.2. Методы для регулирования дорожного движения.....	34
5.3. Методы для анализа дорожного движения.....	36
5.4. Результаты симуляции дорожного движения.....	37
ЗАКЛЮЧЕНИЕ	43
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	45
Приложение А Ссылка на репозиторий с кодом программы и отчетами	48
Приложение Б Алгоритм переключения сигналов светофора в SUMO.....	49

ОСНОВНЫЕ ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

Workflow - это процесс, который включает последовательность действий.

Препроцессинг - это процесс предварительной обработки данных.

CNN – (Convolutional Neural Network) это тип алгоритма глубокого обучения, предназначенный для обработки визуальных данных.

Тензор - это представление данных, которое используется для решения задач сегментации изображений или временных рядов. Например, в задаче семантической сегментации изображений, элементы тензора отвечают о принадлежности каждого пикселя к определённому классу.

Baseline (базовая) архитектура — простая, начальная структура системы (в ML, ПО или ИТ- инфраструктуре), определяющая минимально приемлемый уровень качества или производительности.

Аннотация - процесс добавления к необработанным данным понятных для машинного обучения метаданных (в данном случае добавление контуров объектов на изображении)

Аугментация - искусственное увеличение объема и разнообразия обучающей выборки путем трансформации существующих данных.

SUMO (Simulation of Urban MObility) - бесплатный пакет программного обеспечения с открытым исходным кодом для моделирования транспортных потоков.

Senior Member, IEEE (старший член IEEE) - профессиональный статус, присваиваемый Институтом инженеров электротехники и электроники (IEEE) специалистам, имеющим значительные достижения, опыт работы не менее 10 лет и рекомендации от коллег.

ВВЕДЕНИЕ

Данная работа посвящена исследованию возможностей умных светофоров, основанных на использовании искусственного интеллекта, с целью повышения пропускной способности дорожного движения.

В условиях современного города, пробки, особенно в часы пик, приводят к огромным экономическим, временным, психологическим и иным задержкам. По статистике из-за этой проблемы в крупных мегаполисах теряется около 40% от времени необходимого на поездку [1]. Для стран Европы и США ущерб, производимый скоплением автомобилей, может составлять миллиарды долларов. Например, в США на 2023 ущерб экономике от автомобильных пробок составил \$70,4 млрд [2]. В России, как и в остальных странах мира, проблема пробок так же актуальна. За последние десять лет число легковых автомобилей в России выросло на 8.7 млн. Появление новых развязок и полос движения решает проблему в краткосрочном периоде, но из-за долгих сроков на строительство, стоимости этих работ и постоянно растущего трафика данное решение не является оптимальным в долгосрочной перспективе. Более дешевым решением является модернизация светофоров – простых регуляторов движения, не требующих масштабных перестроек. Современные светофоры работают по трехступенчатой схеме: часы пик, дневное время, ночь, но в случае непредсказуемости потока, данная система становится малоэффективной.

Исходя из исследований ученых пермского политехнического университета, сокращение фаз светофора способствует сокращению заторов на 15-20%, а значит своевременное переключение фаз светофоров будет способствовать более высокой пропускной способности [3]. Существующие адаптивные светофоры частично решают проблему трехступенчатой системы, но они часто работают на устаревших технологиях и сильно зависят от множества физических датчиков, чувствительных к атмосферным условиям, что снижает их эффективность.

Представленное в работе решение заключается в разработке интеллектуальных светофоров, которые будут более устойчивы к внешним влияниям и способны эффективно распознавать транспортные средства и пешеходов. Системы, основанные на искусственном интеллекте, могут самостоятельно анализировать дорожную ситуацию и оперативно переключать сигналы светофора в зависимости от загруженности. Это позволит минимизировать время ожидания на перекрестках и улучшить общую пропускную способность транспортных потоков.

В рамках выпускной квалификационной работы был изучен материал, необходимый для построения архитектуры модели и рассмотрены проекты, решающие похожие задачи.

Был разработан класс, использующий модель YOLO26l-seg, для сегментирования видео с целью определения каждого пикселя на принадлежность к транспортным средствам или людям и получения по собранным данным информации о ситуации на перекрестке. Разработанный класс был протестирован на подготовленных заранее записях дорожного движения. Обучена модель для регулирования дорожного движения. Также, была развернута система симуляции дорожного движения SUMO и проведен анализ полученных после множества симуляций данных.

Процесс выполнения ВКР можно разбить на четыре этапа:

Этап обработки видеопотока – состоит из разработки алгоритма компьютерного зрения для сегментации объектов на видео через модель YOLO и реализует конвейер, в рамках которого полученные сегментационные маски преобразуются в признаки дорожной ситуации и подготавливаются для подачи на вход модели управления потоком.

Этап обучения модели для управления потоком – состоит из обучения модели на заранее подготовленных данных.

Этап моделирования дорожного движения – состоит из разворачивания системы симуляции дорожного движения SUMO.

Заключительный этап анализа предложенного решения – состоит из анализа полученных после множества симуляций данных на светофорах с использованием модели интеллектуального алгоритма и с обычным режимом работы.

Объектом исследования являются существующие процессы управления дорожным движением на регулируемых перекрестках.

Предметом исследования являются интеллектуальные алгоритмы управления сигналами светофоров, на основе данных компьютерного зрения.

Целью выпускной квалификационной работы является разработка интеллектуального алгоритма для переключения сигналов светофора на основе данных, собранных с камер дорожного движения.

Для достижения поставленной цели необходимо было решить следующие задачи:

1. Провести анализ существующих решений для адаптивного управления трафиком.
2. Изучить существующие решения и аналоги по сегментации транспортного трафика с использованием камер.
3. Выбрать Baseline архитектуру.
4. Разработать программу для анализа дорожной ситуации с использованием компьютерного зрения.
5. Выбор и обучение модели на данных, сформированных после работы программы анализа дорожного движения.

6. Развернуть систему контроля трафика и реализовать алгоритм управления сигналами светофора на основе предсказаний модели.
7. Выполнить запуск симуляций с разными параметрами для обычного светофора и для светофора, управляемого нейросетевой моделью.
8. Провести анализ симуляций.

Основная задача – разработка и реализация алгоритма интеллектуального управления сигналами светофора.

Вспомогательная задача – оценка улучшения характеристик движения по сравнению с существующими режимами работы светофора.

Инструменты, использованные при разработке программы: Python, JupiterLab, Roboflow, SUMO.

1. АНАЛИЗ СУЩЕСТВУЮЩИХ РЕШЕНИЙ ДЛЯ АДАПТИВНОГО УПРАВЛЕНИЯ ТРАФИКОМ

Далее будут рассмотрены:

- адаптивный светофор,
- использование ML для детекции автомобилей в России,
- использование светофоров, управляемых ИИ, в странах Европы.

1.1. Адаптивный светофор

Адаптивные светофоры используют устаревшую технологию подсчета автомобилей на перекрестках, а именно физические датчики или видеодетектор фона в определенных зонах. Недостаток этой технологии заключается в том, что обнаружение транспорта с помощью датчиков ограничено несколькими метрами перед переходом, а значит видеодетектор фона показывает заполненность только этого участка. К тому же, если камера не сможет корректно различить эту область на видео из-за расстояния, смещения или погодных условий, то светофор перестанет работать оптимально. Пример адаптивного светофора представлен на рисунке 1.

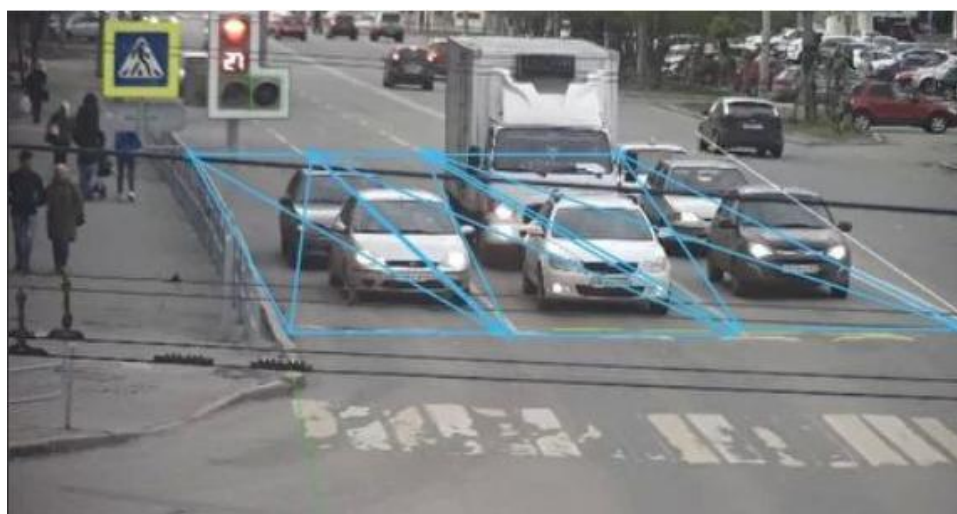


Рис 1. Видеодетектор адаптивного светофора

Помимо устаревших технологий, установка физических датчиков для зоны детекции требует реконструкции дорожного полотна и местами стоимость установки может исчисляться миллионами [4 - 5].

Таким образом, адаптивные светофоры работают только в ситуациях, где правила переключения абсолютно очевидны или на мало загруженных перекрестках.

1.2. Использование ML для детекции автомобилей в России

Преимущества нейросетевых светофоров над адаптивными:

- не нуждаются в установке физических датчиков,
- получают более информативные данные, например, тип транспорта, его длина, его ID,
- помимо транспортных средств, могут учитывать скопления пешеходов,
- количество объектов для детекции не ограничивается только типом транспорт и пешеход.

Рассмотренный в предыдущем пункте адаптивный светофор, хоть и умеет переключать сигналы светофора за счет датчиков, но делает это без использования компьютерного зрения. Детекция автомобилей в России используется в основном для фиксации нарушений на дорогах.

Далее будут рассмотрены проекты из России использующие компьютерное зрение для контроля движения на дорогах:

- видеодетектор автотранспортных средств ИНФОПРО и видеодетектор транспорта РТК,
- НПО ИТС: Паук Трафик,

1.2.1. Видеодетектор автотранспортных средств ИНФОПРО и Видеодетектор транспорта РТК

Оба проекта работают по принципу адаптивных светофоров, но используют нейросетевые алгоритмы для вычисления геометрического центра транспортного средства. Особенности этих проектов являются получение и анализ статистики по следующим параметрам: объем потока, средняя скорость, занятость дороги, интервал следования автотранспортных средств [6]. Система ИНФОПРО, может переключаться в режим адаптивного светофора [7], когда Видеодетектор РТК может определять класс транспортного средства. На рисунке 2 представлен интерфейс Видеодетектора РТК, а на рисунке 3 видеодетектор ИНФОПРО.

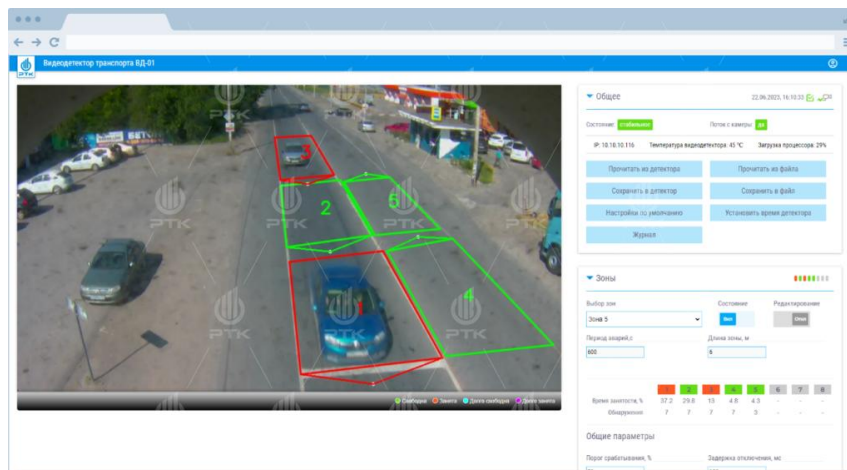


Рис 2. Видеодетектор РТК

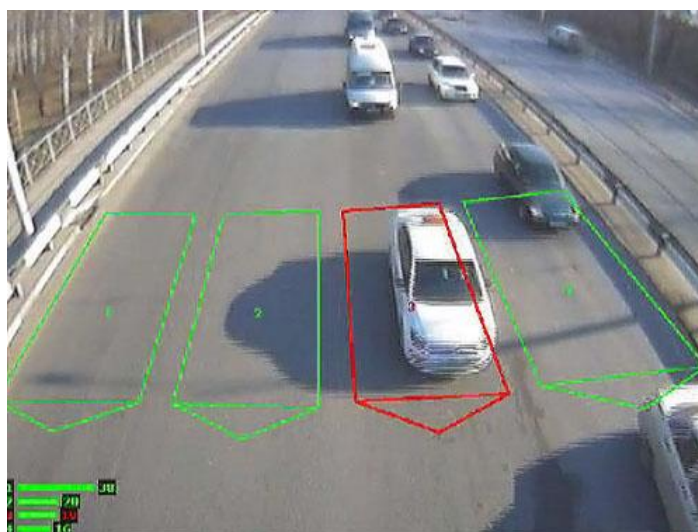


Рис 3. Видеодетектор ИНФОПРО

1.2.2. НПО ИТС: Паук Трафик

Проект НПО ИТС: Паук Трафик тоже используется для аналитики дорожного движения, но в отличие от рассмотренных ранее проектов активно использует технологии компьютерного зрения. Видеодетектор способен определять количество автомобилей, плотность трафика, среднюю скорость на дороге и индивидуальную скорость транспортного средства, а также его тип. Разработанный инженерами «НПО «ИТС» детектор устойчив к погодным условиям и может контролировать движение даже на многополосных магистралях с погрешностью детектирования не выше 5%. Помимо анализа и сбора данных, комплекс видеофиксации «ПАУК» выявляет факты нарушения скоростного режима, проезда на запрещающий сигнал светофора, правил пользования ремнями безопасности и телефоном во время движения, пересечение стоп-линии и другие нарушения ПДД, а также умеет распознавать госномера нарушителей [8]. На рисунке 4 представлен видеодетектор Паук Трафик.

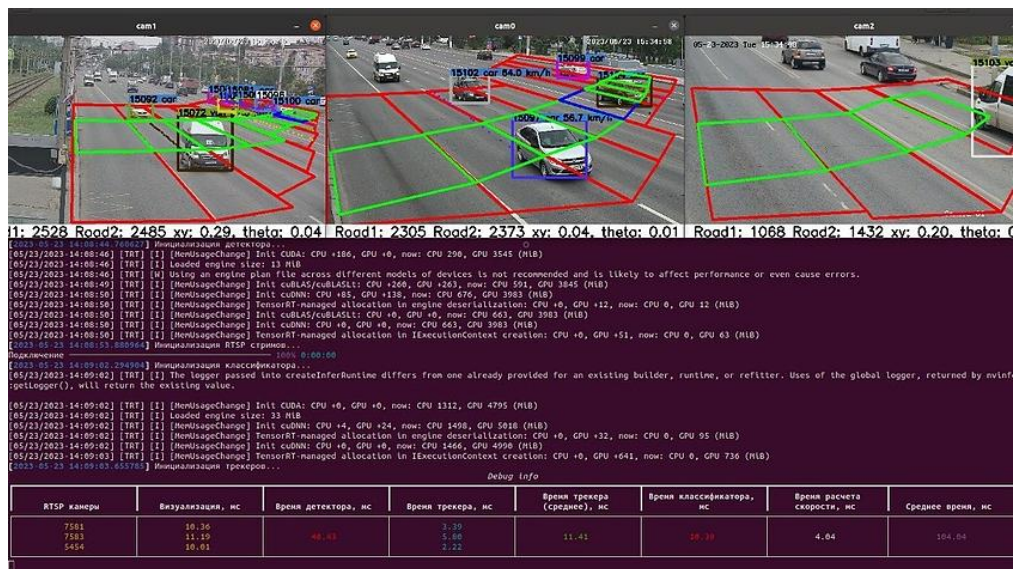


Рис 4. Видеодетектор Паук Трафик

1.3. Использование светофоров, управляемых ИИ, в странах Европы

В отличие от, Российских проектов с использованием Компьютерного зрения, для анализа дорожного движения, в Германии уже проводятся испытания светофоров с искусственным интеллектом.

Так в 2023 году в немецком городе Лемго проект KI4LSA установил светофоры с камерами высокого разрешения и радарными датчиками, для фиксирования реальной дорожной ситуации, включая среднюю скорость автомобилей и время ожидания, для интеллектуального прогнозирующего переключения светофоров [9 - 10].

В конце наблюдений, использование искусственного интеллекта улучшило транспортный поток на 10-15% [11]. На рисунке 5 представлен видеодетектор KI4LSA.



Рис 5. Видеодетектор KI4LSA

2. Анализ архитектуры

Далее будут рассмотрены:

- Выбор архитектуры Baseline,
- Ultralytics YOLO,
- CatBoostClassifier,

2.1. Выбор архитектуры Baseline

Базовая архитектура для будет основана на архитектуре, разработанной в 2024 году старшими членами IEEE в рамках исследовательской работы «Edge ML Technique for Smart Traffic Management in Intelligent Transportation Systems» [12].

Архитектура, представленная на рисунке 6 состоит из 3 частей:

1. Сегментация кадров видеоряда.
2. Полученный на предыдущем шаге кадр обрабатывается с помощью обученной модели YOLO для сегментации объектов (машин).
3. Количество машин передается в алгоритм для переключения светофора, который выводит сигнал светофора, для переключения.

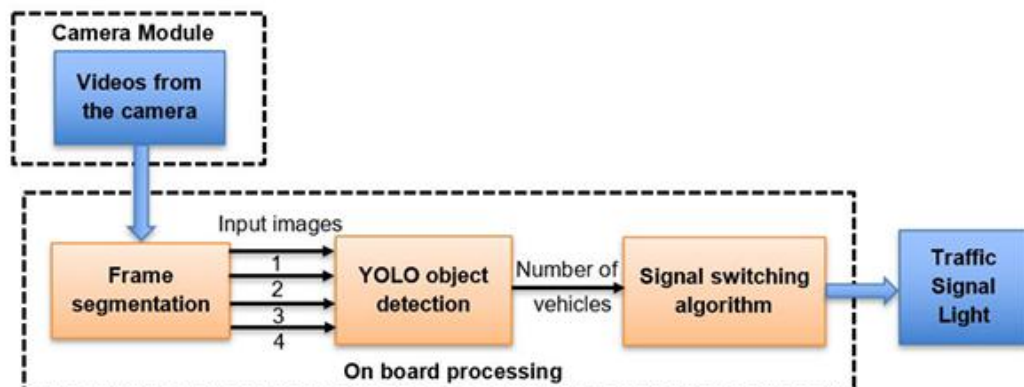


Рис 6. Baseline архитектура

В рамках представленного исследования для детекции автомобилей использовалась модель YOLOv3. Для детекции автомобилей в данной работе используется модель YOLOv26, т.к. по представленным метрикам на официальном сайте она показывает лучшие результаты [13].

В Baseline архитектуре, для получения сигнала светофора используется комплексный алгоритм. В рамках выпускной квалификационной работы для регулирования продолжительности сигналов будет использоваться CatBoostClassifier.

2.2. Ultralytics YOLO

Задача сегментации – одна из ключевых задач в области компьютерного зрения, делится на три группы [14]:

- семантическая сегментация, тип сегментации, классифицирующий каждый пиксель изображения, без возможности определить разные экземпляры одного и того же объекта (рисунок 7);



Рис 7. Пример семантической сегментации

- instance segmentation, тип сегментации, способный различать не только класс объекта, но и различные экземпляры (рисунок 8);



Рис 8. Пример instance сегментации

- паноптическая сегментация, распознает не только отдельные экземпляры объектов, но и фоновые (небо/земля), что позволяет более комплексно анализировать сцены (рисунок 9).

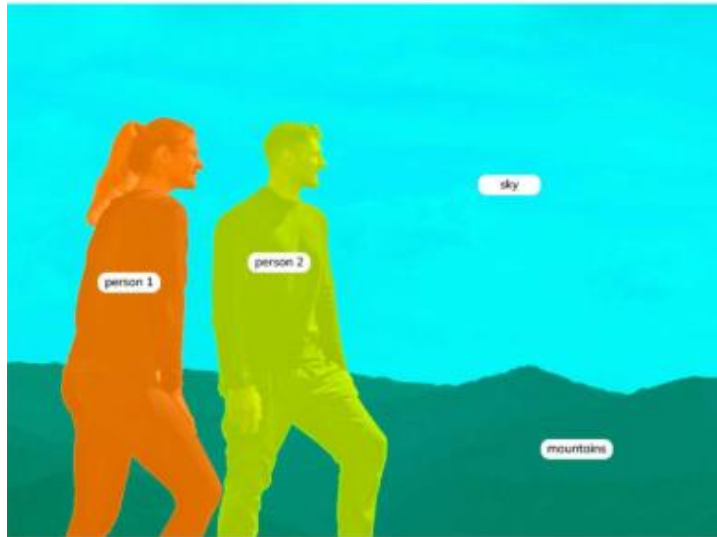


Рис 9. Пример паноптической сегментации

В текущей работе, для определения объектов и их наличия будет использоваться метод Instance Segmentation.

Платформа ultralytics YOLO (You Only Look Once) специализируется на быстром и точном обнаружении объектов, а также поддерживает задачи сегментации. Помимо быстрого и точного определения объектов, система указывает количество экземпляров, обнаруженных на картинке, что необходимо для решения текущей задачи.

В рамках Выпускной квалификационной работы использовалась YOLOv26 с трехсекционной архитектурой Backbone, Neck и Head [15], представленной на рисунке 10, структура которой идентична архитектуре YOLO v11.

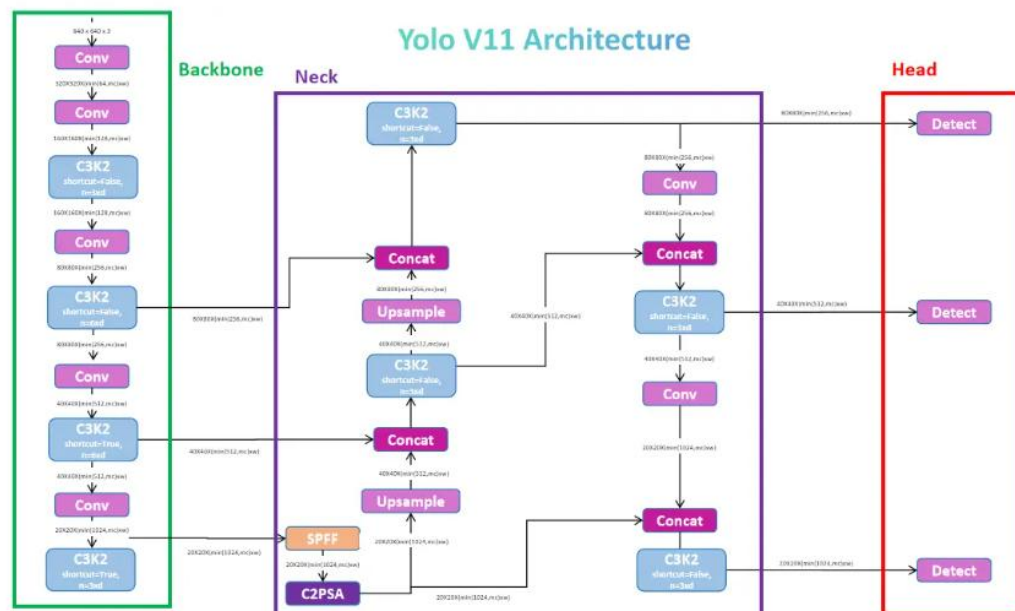


Рис 10. Архитектура YOLO v11 (v26)

Backbone часть извлекает из изображения полезные признаки на основе эффективного “bottleneck-based” алгоритма.

Neck – обрабатывает выходные данные Backbone и передает полученные признаки головной части

Head (головная) часть, работает исходя из поставленной задачи:

- обнаружение объектов на изображении,
- семантическая сегментация объектов,
- определение ключевых точек для позы (например, на человеке голова, локти и т.д.),
- классификация изображений и т.д.

Bottleneck-based слой представляет собой два сверточных блока, соединенных последовательно с функцией конкатенации, если параметр `shortcut = true`, входные параметры объединяются с выходными параметрами второго сверточного слоя. Такая структура повышает эффективность и процесс обучения. На рисунке 11 представлен Bottleneck слой.

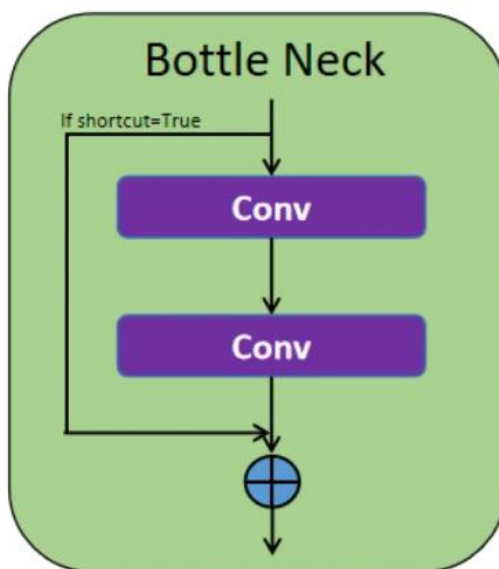


Рис 11. Bottleneck слой

2.3. CatBoostClassifier

CatBoostClassifier – класс из библиотеки машинного обучения CatBoost, предназначенный для задач бинарной и мультиклассовой классификации. CatBoost – алгоритм градиентного спуска, разработанный командой Яндекса [16]. В качестве классификатора был выбран из-за возможности мультиклассовой классификации и множества дополнительных функций, упрощающих работу. Например, обработка категориальных признаков позволяет указывать не числовые признаки без предварительной их обработки.

3. ПРОГРАММА ДЛЯ АНАЛИЗА ДОРОЖНОЙ СИТУАЦИИ С ИСПОЛЬЗОВАНИЕМ КОМПЬЮТЕРНОГО ЗРЕНИЯ

Будут рассмотрены:

- поиск и разметка датасета,
- результаты дообучения модели сегментации,
- реализация класса, для сегментации видеоряда и получения информации о трафике.

В качестве результатов будут представлены видео, демонстрирующие работу программы.

3.1. Поиск и разметка датасета

В рамках работы был написан класс *VideoConverterClassV2* на языке python для сегментации объектов в видеопотоке с использованием предобученной модели YOLO11l-seg. Модели Yolo для сегментации делятся на n, s, m, l, x, различия между моделями заключаются в скорости и точности детекции. Если необходима скорость детекции, но не ее точность выбирается модель n, иначе, если необходима точность, но не скорость выбирается модель x [17].

На рисунке 12 представлен график точности, по результатам обучения модели на 100 эпохах и с использованием данных из датасета COCO, взятого из открытого доступа.



Рис 12. График точности обучения модели

Для улучшения точности сегментации для текущей задачи модель необходимо дообучить на специфичных данных. Для обучения модели необходимо иметь датасет с размеченными данными. В результате тщательного анализа сайтов и платформ, связанных

с машинным обучением, подходящих размеченных данных с камер наблюдения обнаружено не было. В связи с этим, в качестве альтернативного подхода рассматривалась возможность использования прямых трансляций улиц городов в России, для ручной разметки видеоматериала. Есть множество сайтов с прямыми трансляциями частных камер установленных по городу. Некоторые камеры установлены на перекрестках и рядом со светофорами. Такие камеры можно использовать, как для обучения, так и для тестирования распознавания. Также можно запросить доступ к видеозаписи с муниципальных камер, интегрированных в единый центр хранения и обработки данных, обратившись в необходимые правоохранительные органы [18].

Исходя из результатов анализа доступных источников, было решено остановиться на втором варианте. Для создания датасета использовался онлайн сервис Roboflow, а в качестве данных для разметки - онлайн трансляции с сайта <https://setitagila.ru/>.

Roboflow - предоставляет инструменты для разметки данных и экспорта наборов данных в различных форматах, включая YOLO [19]. Для разметки данных был выбран именно он, так как среди существующих инструментов аннотирования является самым доступным.

На шаге разметки изображений были добавлены классы для автобусных остановок, велосипедистов, дорожных знаков и т.д. Процесс аннотации представлен на рисунке 13.

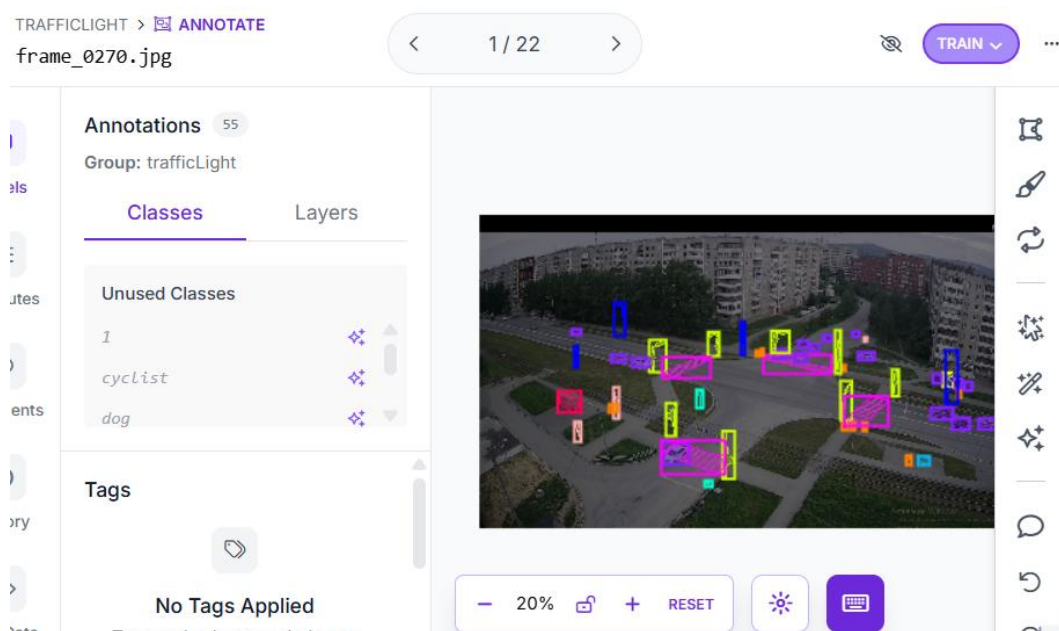


Рис 13. Процесс аннотации

По результатам аннотации 22 изображений, датасет был разделен на тренировочный и валидирующий в соотношении 80/20. Также, на этапе аугментации с помощью добавления шумов на изображения и изменения угла поворота получилось увеличить объем данных для обучения до 58 (рисунок 14).

5

Create 

Review your selections and select a version size to create a moment-in-time snapshot of your dataset with the applied transformations.

Larger versions take longer to train but often result in better model performance. [See how this is calculated](#)

Maximum Version Size

58 images (3x)



Рис 14. Процесс аугментации датасета.

3.2. Результаты дообучения модели сегментации

На рисунке 15 представлен график точности после дообучения модели на 100 эпохах с использованием нового датасета.

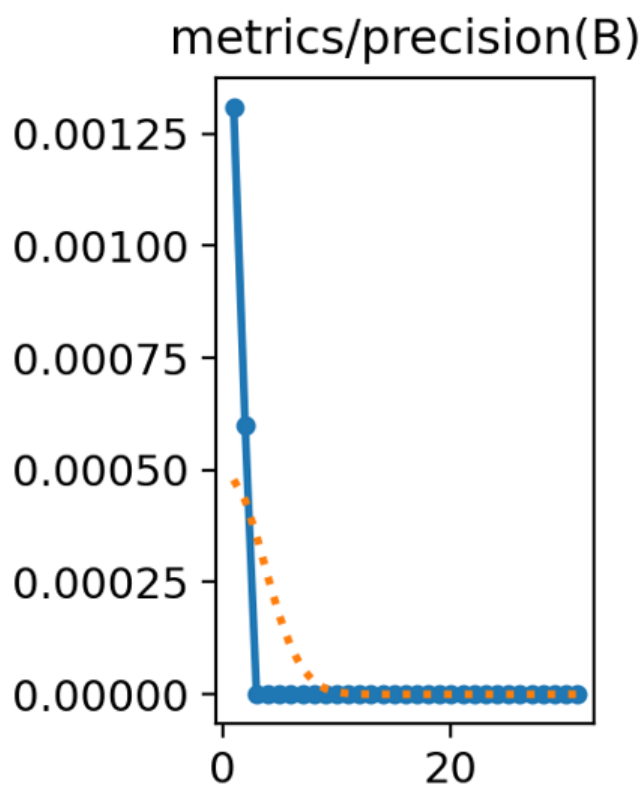


Рис 15. График точности обучения модели.

После анализа изображений для обучения были обнаружены криво размеченные объекты, например, на рисунке 16 объект автомобиля (car) растянулся в верхний левый угол, что могло стать причиной неудовлетворительным результатам.

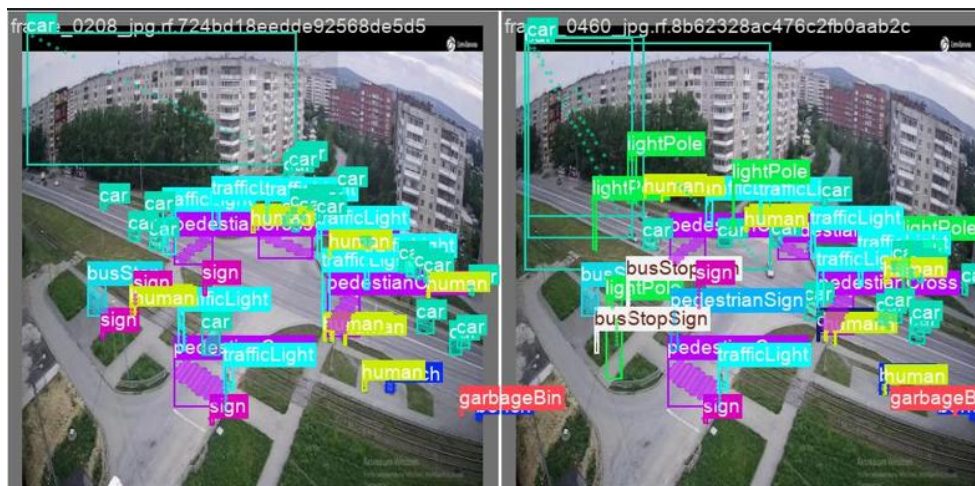


Рис 16. Батч, использовавшийся для обучения модели.

По итогу было решено использовать предобученную модель до ее дообучения на новых данных. На рисунке 17 представлено сравнение результата сегментации объектов на предобученной модели и модели полученной в результате дообучения.

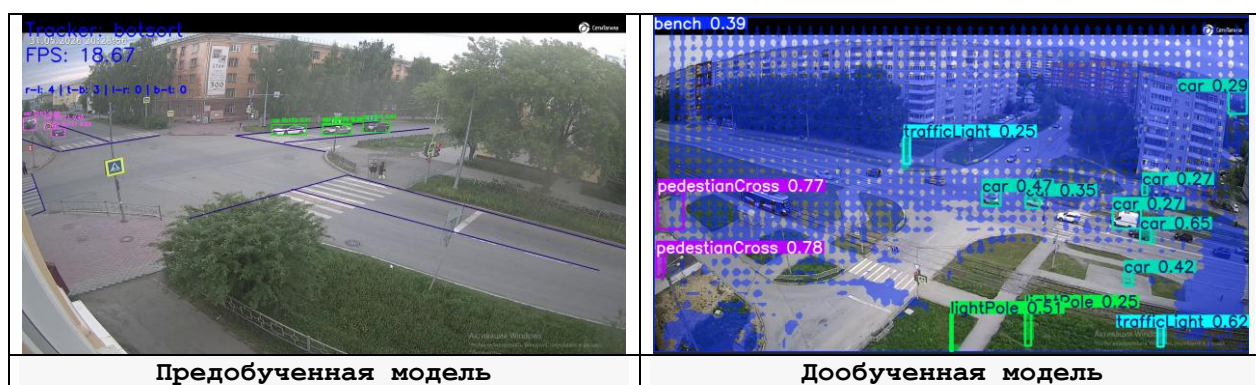


Рис 17. Результат сегментации на 2-х моделях.

3.3. Реализация класса, для сегментации видеоряда и получения информации о трафике

Класс VideoConverterClassV2 был написан для работы с видео.

Будут рассмотрены:

- API-методы для работы с видео,
- процесс реализации класса,
- результаты работы класса.

Процесс реализации класса можно разбить на 3 этапа:

- методы для сегментации видео,
- алгоритм для подсчета объектов на светофоре,
- выбор алгоритма для отслеживания объектов.

3.3.1. API-методы для работы с видео

Для работы с видео использовалась библиотека CV2. Cv2 это модуль OpenCV в Python - открытой библиотеки для работы с алгоритмами компьютерного зрения [20]. В рамках работы используется для преобразований изображений и видео, а также для задания областей наблюдения за автомобилями. Методы для работы с видеорядом, использовавшиеся во время разработки:

- `cv2.VideoCapture(fileName)` – функция для захвата, видео записывается в переменную `cap`, на вход подается путь к файлу, далее в цикле `while True`: с помощью `cap.read()` считывается кадр видео, данная функция возвращает 2 значения (логическое значение корректности считывания кадра и сам кадр);
- `cap.get(cv2.CAP_PROP_FPS)` – используется для получения кадров в секунду у обрабатываемого видео, `fps` (frames per second) понадобятся для записи полученных изображений в видео;
- `cv2.waitKey(1)` - функция, прерывающая выполнение бесконечного цикла, ждет нажатия клавиши 1 миллисекунду и проверяет на необходимость освободить захват видео с помощью `cv2.destroyAllWindows()`, чтобы выйти из цикла;
- `cv2.VideoWriter(pathToVideo, cv2.VideoWriter_fourcc(*'DIVX'), fps, (width, height))` - функция для записи видео в файл, на вход подается путь до файла для сохранения, переменная `fourcc` - 4-байтовый код, который используется для указания видекодека, количество кадров в секунду и размер видео в пикселях;
- `video.write(cv2.imread(os.path.join(pathToVideoImages, image)))` – используется для конвертации папки изображений в видеоряд, в рамках работы программы, во время сегментации, обработанные изображения сохраняются в папку, чтобы в случае вылета программы полученный результат можно было склеить в один видеоряд;
- `setMouseCallback(cv2.setMouseCallback('image', self.extract_coordinates))` – функция для получения и обработки координат клика по фрейму с видеорядом

3.3.2. Методы для сегментации видео

Класс *VideoConverterClassV2* состоит из двух основных методов:

- `maskVideoImgs` – с использованием библиотеки `cv2`, для захвата видеопотока, алгоритм обрабатывает каждый видеокادر и записывает в отдельную папку
- `convertToVid` - метод, конвертирует все обработанные кадры из папки в одно видео с использованием `cv2`.

3.3.3. Алгоритм для подсчета объектов на светофоре

Для подсчета количества машин на светофорах пользователю необходимо определить область светофора в которой совершается подсчет вручную, для этого был реализован класс DrawLineWidget для записи координат линии в массив с помощью callback функции *extract_coordinates*, функция представлена на рисунке 18.

```
def extract_coordinates(self, event, x, y, flags, parameters):
    # Record starting (x,y) coordinates on left mouse button click
    if event == cv2.EVENT_LBUTTONDOWN:
        if (len(self.image_coordinates) < 2):
            self.image_coordinates = [(x,y)]

    # Record ending (x,y) coordinates on left mouse button release
    elif event == cv2.EVENT_LBUTTONUP:
        if (len(self.image_coordinates) < 2):
            self.image_coordinates.append((x,y))
            print('Starting: {}, Ending: {}'.format(self.image_coordinates[0], self.image_coordinates[1]))

            # Draw line
            drawLine(self.clone, self.image_coordinates)
            cv2.imshow("image", self.clone)

    # Clear drawing boxes on right mouse button click
    elif event == cv2.EVENT_RBUTTONDOWN:
        self.clone = self.original_image.copy()
        self.image_coordinates = []
```

Рис 18. Метод для записи координат линии в массив.

Полученные линии добавляются в массивы, соответствующие зонам перед светофором после чего записываются в переменные класса AreaCoords, хранящего в себе информацию о точках области. Структура класса AreaCoords представлена на рисунке 19.

```
class AreaCoords:
    def __init__(self, top_left=None, top_right=None, bottom_right=None, bottom_left=None):
        self.top_left = top_left
        self.top_right = top_right
        self.bottom_right = bottom_right
        self.bottom_left = bottom_left

    def set_coords(self, top_left=None, top_right=None, bottom_right=None, bottom_left=None):
        if top_left != None:
            self.top_left = top_left
        if top_right != None:
            self.top_right = top_right
        if bottom_right != None:
            self.bottom_right = bottom_right
        if bottom_left != None:
            self.bottom_left = bottom_left

    def is_any_none(self):
        """
        Есть ли хоть одна координата None
        """
        return True if self.top_left == None or self.top_right == None or self.bottom_left == None or self.bottom_right == None else False

    def get_area_coords(self):
        """
        Returns array of coords
        """
        return np.array([self.top_left, self.top_right, self.bottom_right, self.bottom_left])
```

Рис 19. Класс для хранения информации об области перед светофором

Области, нарисованные пользователем, сохраняются в json файл, чтобы при повторном запуске не повторять процесс разметки.

Далее из результата сегментации извлекаются границы объектов и с помощью метода *isIntersectOrInside* происходит проверка на наличие общих точек у квадрата границы

объекта и областей светофора. Для проверки на пересечения используется библиотека `shapely.geometry`. Функция `isIntersectOrInside` представлена на рисунке 20.

```
from shapely.geometry import Polygon

def isIntersectOrInside(square_coords, area_coords):
    """
    Проверяет есть ли общие точки у квадрата и области
    square_coords - координаты квадрата
    area_coords - координаты области
    """
    square = Polygon(square_coords)
    target_area = Polygon(area_coords)
    # 1. Проверить, пересекает ли квадрат область (есть ли общие точки)
    if square.intersects(target_area):
        return True
    # 2. Проверить, находится ли квадрат ПОЛНОСТЬЮ внутри области
    if square.within(target_area):
        return True
    return False
```

Рис 20. Метод проверяющий на наличие общих точек квадрата и области.

Чтобы точнее определить принадлежность объекта к наблюдаемой области применяется метод `isLineInsideCorrectArea`, который проверяет, если крайняя граница объекта к встречному движению пересекает либо находится в нужном от него направлении. Метод представлен на рисунке 21.

```
def isLineInsideCorrectArea(main_line: LineString, object_line: LineString, correct_position: Direction, withIntersect = True) -> str:
    """
    Проверяет, если крайняя граница объекта к встречному движению пересекает либо находится в нужном от него направлении
    (для движения справа налево, проверяем нижнюю границу объекта)
    main_line - линия направления движения (координаты передаются по направлению движения)
    object_line - крайняя линия объекта
    correct_position - требуемое расположение линии объекта относительно основной
    returns: boolean
    """
    if withIntersect and object_line.intersects(main_line):
        return True
    # Смещаем на 1 единицу влево и вправо
    left = main_line.offset_curve(1.0)
    right = main_line.offset_curve(-1.0)

    # Сравниваем расстояния
    position = Direction.LEFT if left.distance(object_line) < right.distance(object_line) else Direction.RIGHT
    return position == correct_position
```

Рис 21. Более точный метод, проверяющий на наличие общих точек квадрата и области.

Алгоритм для определения принадлежности объекта к наблюдаемым зонам представлен на рисунке 22.

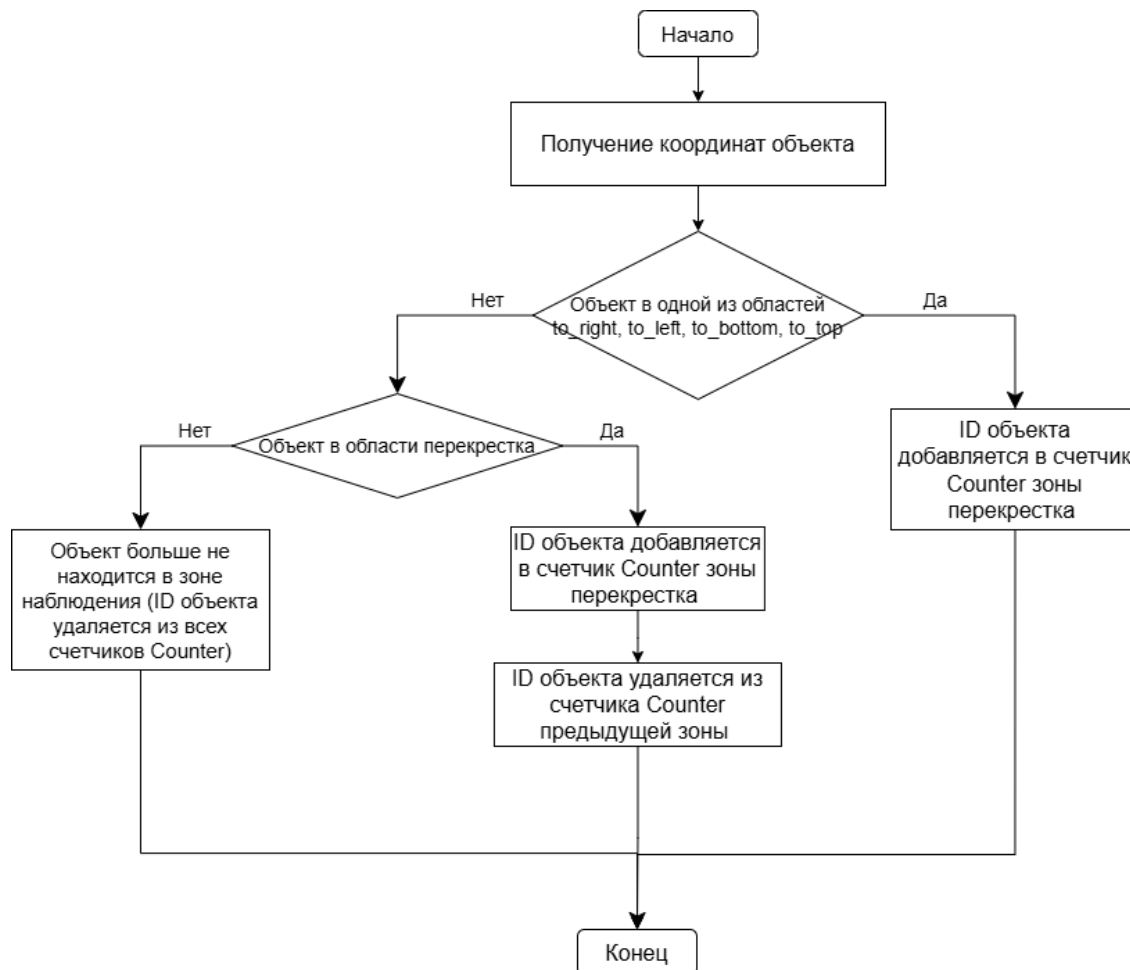


Рис 22. Алгоритм для определения принадлежности объекта к наблюдаемым зонам.

Суть алгоритма заключается в том, чтобы добавлять ID объекта, полученного после сегментации кадра, в один из счетчиков типа Counter (Counter – счетчик уникальных значений библиотеки collection) для каждой из областей движения трафика: вправо, влево, вверх, вниз, на пересечении (область перекрестка). Если ID объекта пропадает из всех областей, считается, что объекта выехал из зоны перекрестка. Счетчик обновляется, если машина остается в наблюдаемой зоне после изменения фазы светофора.

После того, как алгоритм прошелся по всем обнаруженным объектам, в переменную типа dataframe добавляется строка о ситуации на дороге в текущий момент. Часть кода, отвечающая за сбор информации, представлена на рисунке 23.


```

# записываем информацию о ситуации на дороге в dataframe
info_collected = {'time_of_info': curIm,
                  'count_to_r': count_left_right,
                  'count_to_l': count_right_left,
                  'count_to_b': count_top_bottom,
                  'count_to_t': count_bottom_top,
                  'count_stuck_middle': self.getTrullyStuck(_counter=self.objects_stuck_middle),
                  'count_stuck_to_r': self.getTrullyStuck(_counter=self.objects_stuck_to_r),
                  'count_stuck_to_l': self.getTrullyStuck(_counter=self.objects_stuck_to_l),
                  'count_stuck_to_b': self.getTrullyStuck(_counter=self.objects_stuck_to_b),
                  'count_stuck_to_t': self.getTrullyStuck(_counter=self.objects_stuck_to_t),
                  'is_jam_next_l': 0,
                  'is_jam_next_r': 0,
                  'is_jam_next_t': 0,
                  'is_jam_next_b': 0,
                  'is_green_horizontal': 1 if isSwitchFrameX2 else 0,
                  'is_green_vertical': 0 if isSwitchFrameX2 else 1,
                  'y': 1,
                  }
dfInfo = pd.DataFrame(info_collected, index=[0])
self.trafficInfoDF = pd.concat([self.trafficInfoDF, dfInfo], ignore_index=True)

```

Рис 23. Часть кода, отвечающая за сбор информации на перекрестке.

В столбики 'count_to_...' записывается количество машин в текущий момент для каждого из направлений. В столбики 'count_stuck_...' записывается количество машин, которые стоят в области более 2 фаз сигнала светофора. В столбики 'is_jam_next_...' записывается информация о наличии пробки на следующем в направлении светофоре. В столбики 'is_green_...' записывается для какого направления горит светофор.

Про столбики 'is_jam_...' и 'count_stuck_middle' подробнее будет расписано в пункте 4.

3.3.4. Выбор алгоритма для отслеживания объектов

Для сегментации видеопотока модели Yolo предоставляют на выбор алгоритмы для многообъектного трекинга BoT-SORT (Bag of Freebies SORT) и ByteTrack [21]. ByteTrack – алгоритм отслеживания объектов в видеопотоках, способный быстро определять объекты на кадре, устойчив к частичным перекрытиям и исчезновениям на короткое время. Вместо того чтобы отбрасывать размытые или слабо заметные объекты (например, при перекрытии другим объектом), он сохраняет их и связывает с траекториями [22]. BoT-SORT – более сложный и точный алгоритм, который лучше справляется с изменением траекторий движения объектов и устойчив к более долгим исчезновениям объектов, объект не «теряется из виду», а всегда остается опознанным даже в рамках большого количества предметов [22].

В таблице 1 представлены результаты запуска сегментации объектов на видеопотоке с использованием двух алгоритмов.

Таблица 1

Результаты запуска сегментации объектов на видеопотоке
с использованием BoT-SORT и ByteTrack.

	BoT-SORT	ByteTrack	BoT-SORT (без вывода видео)	ByteTrack (без вывода видео)
FPS (кадр/сек)	17.25	26	16.76	29.32
Время выполнения (сек)	93.21	69.04	94.92	63.93

По результатам нескольких запусков ByteTrack выполнил задачу почти в 2 раза быстрее BoT-SORT. Отключение отображения процесса сегментации (вывода на видео) не повлияла на скорость процесса. Однако к концу сегментации ByteTrack насчитал 841 объект, на 30 больше объектов чем BoT-SORT. Пример ID объектов на алгоритме BoT-SORT и ByteTrack представлен на рисунке 24.



Рис 24. Пример Id объектов с использованием двух алгоритмов.

Таким образом, несмотря на скорость сегментации объектов, в рамках работы использовался алгоритм BoT-SORT.

3.3.5. Результаты работы класса

С использованием разработанного класса была проведена обработка нескольких видео.

Демонстрация работы класса представлена на рисунке 25.



Рис 25. Обработанные программой кадры с разных видео.

Видео с сегментацией можно найти, перейдя по ссылке из приложения А.

4. МОДЕЛЬ ДЛЯ ПРЕДСКАЗАНИЯ СИГНАЛА СВЕТОФОРА

Будут рассмотрены:

- создание датасета для обучения,
- обучение модели и результаты метрик.

В качестве результатов будут представлены метрики обучения модели и видео, демонстрирующие работу программы предсказания сигнала светофора.

4.1. Создание датасета для обучения

Для создания датасета использовались данные собранные после сегментации видео с использованием программы из прошлого пункта. Размер результирующего датасета составлял 3000 строк. Чтобы увеличить размерность, было сгенерировано дополнительно 65000 строк. Для новых строк случайным образом заполнялись признаки 'is_jam_next...'. Также, чтобы модель умела обрабатывать редкие ситуации, к датасету добавились случаи, когда середина перекрестка забита.

После формирования датасета, с помощью функции `get_Y(DataFrame)` для каждой строки обновлялся параметр 'y'. Функция определяет полезность включенного сигнала для каждого из направлений. Алгоритм для определения полезности любого из направлений представлен на рисунке 26.

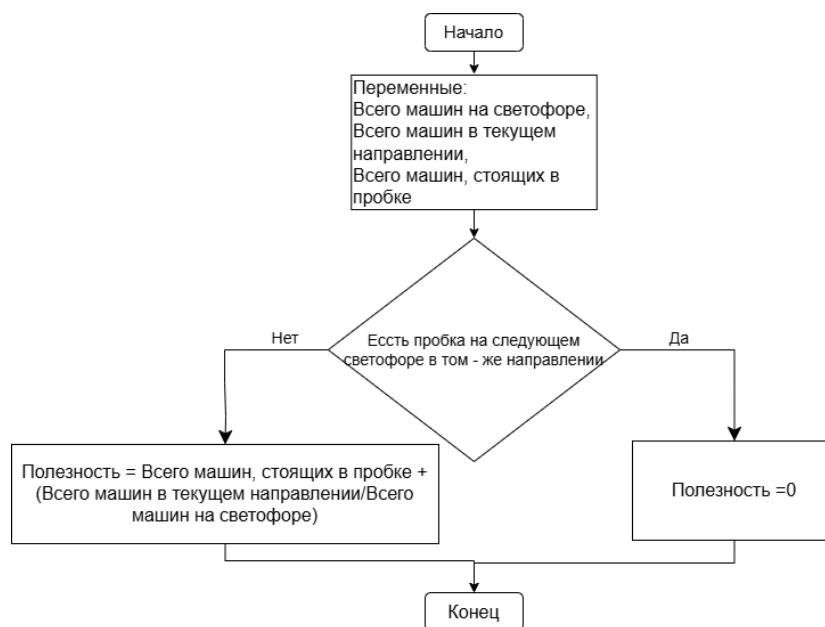


Рис 26. Алгоритм вычисления полезности переключения на зеленый сигнал для направления движения.

Алгоритм проверяет на наличие пробки на следующем светофоре в том же направлении. Если пробка есть светофор не должен пускать в выбранном направлении

новые машины, пока пробка не пропадет. Иначе, вычисляется сумма доли машин в выбранном направлении от общего количества машин в зоне наблюдения с количеством машин, стоящих в пробке в этом же направлении, в текущий момент. Приоритет у машин в пробке больше, значит и полезность этого направления должна увеличиться. Также, для всех направлений учитывается признак пробки посреди перекрестка ('count_stuck_middle'). Пока пробка посреди перекрестка не исчезнет, для всех направлений должен гореть красный цвет. По итогу алгоритма признак 'у' может быть: 0 – включить красный для всех направлений, 1 – включить зеленый для вертикального направления, 2 – включить зеленый для горизонтального направления. Реализация алгоритма для получения сигнала светофора представлена на рисунке 27.

```
# Считаем Полезности включенного сигнала для каждого из направлений
# доля от всех машин в каждом направлении +
# + Количество машин которые стоят несколько фаз в каждом из направлений
# (Для таких машин приоритет выше)

# В случае, если по направлению вправо нету пробки
df_c['utility_r'] = np.where(df_c['is_jam_next_r'] == 0,
                             (df_c['count_to_r'] / df_c['cars_count']) + df_c['count_stuck_to_r'],
                             0)
df_c['utility_l'] = np.where(df_c['is_jam_next_l'] == 0,
                             (df_c['count_to_l'] / df_c['cars_count']) + df_c['count_stuck_to_l'],
                             0)
df_c['utility_t'] = np.where(df_c['is_jam_next_t'] == 0,
                             (df_c['count_to_t'] / df_c['cars_count']) + df_c['count_stuck_to_t'],
                             0)
df_c['utility_b'] = np.where(df_c['is_jam_next_b'] == 0,
                             (df_c['count_to_b'] / df_c['cars_count']) + df_c['count_stuck_to_b'],
                             0)

# print(df_c.head())

# решаем на основе полезности фаз с учётом пробок на следующих светофорах, какому направлению давать сигнал
utilityComaprison = (df_c['utility_r'] + df_c['utility_l']) > (df_c['utility_t'] + df_c['utility_b'])

conditions = [
    (stuckMiddleCondition == True),
    (utilityComaprison == True),
    (utilityComaprison != True)
]
# choices = [DirectionOfGreen.NONE, DirectionOfGreen.HORIZONTAL, DirectionOfGreen.VERTICAL]
choices = [0, 2, 1]
# temp_series = np.select(conditions, choices, default=0)
# return pd.Series(temp_series).map({0: DirectionOfGreen.NONE, 2: DirectionOfGreen.HORIZONTAL, 1: DirectionOfGreen.VERTICAL})
return np.select(conditions, choices, default=0)
```

Рис 27. Реализация алгоритма, для вычисления полезности переключения на зеленый сигнал для направления движения.

4.2. Обучение модели и результаты метрик

Модель обучалась на 500 итерациях. В качестве функции потерь была выбрана MultiClass, так как модель может предсказать 3 состояния светофора. В качестве категориальных признаков были указаны столбцы, указывающие на наличие пробки на следующем светофоре в каждом из направлений is_jam_next_t, is_jam_next_b, is_jam_next_l, is_jam_next_r. Результаты обучения представлены на рисунке 28.

```

0:    learn: 0.8871201    test: 0.8865414 best: 0.8865414 (0)    total: 6.79ms    remaining: 3.39s
50:   learn: 0.9745216    test: 0.9731706 best: 0.9731706 (50)   total: 265ms    remaining: 2.33s
100:  learn: 0.9849579    test: 0.9835514 best: 0.9835514 (100)  total: 535ms    remaining: 2.11s
150:  learn: 0.9895820    test: 0.9868411 best: 0.9870605 (144)  total: 807ms    remaining: 1.86s
200:  learn: 0.9922687    test: 0.9898384 best: 0.9902040 (198)  total: 1.08s    remaining: 1.61s
250:  learn: 0.9941330    test: 0.9912274 best: 0.9915198 (240)  total: 1.35s    remaining: 1.34s
300:  learn: 0.9953759    test: 0.9921047 best: 0.9922509 (289)  total: 1.58s    remaining: 1.04s
350:  learn: 0.9964725    test: 0.9926164 best: 0.9929088 (340)  total: 1.81s    remaining: 769ms
400:  learn: 0.9971305    test: 0.9930550 best: 0.9932013 (380)  total: 2.05s    remaining: 506ms
450:  learn: 0.9973498    test: 0.9929819 best: 0.9933475 (426)  total: 2.28s    remaining: 247ms
499:  learn: 0.9977519    test: 0.9934206 best: 0.9934206 (499)  total: 2.5s     remaining: 0us

bestTest = 0.9934205717
bestIteration = 499

      precision    recall  f1-score   support

0         1.00       1.00       1.00       5603
1         0.99       0.99       0.99       4283
2         0.99       0.99       0.99       3793

```

Рис 28. Результаты обучения модели на собранных данных.

По результатам обучения можно сделать вывод, что точность предсказания сигнала светофора практически идеальная для каждого из классов. В случае с переключением режима на “Красный для всех” точность предсказания 100%, это связано с тем, что такой режим активен, только если в столбике ‘count_stuck_middle’ есть любое значение больше 0. На рисунке 29 представлена значимость каждого из признаков, где можно заметить, что столбик, отвечающий за переключение в режим 0 имеет наибольшую значимость, а признаки текущего сигнала на светофоре ‘is_green_...’ практически не влияют на результат.

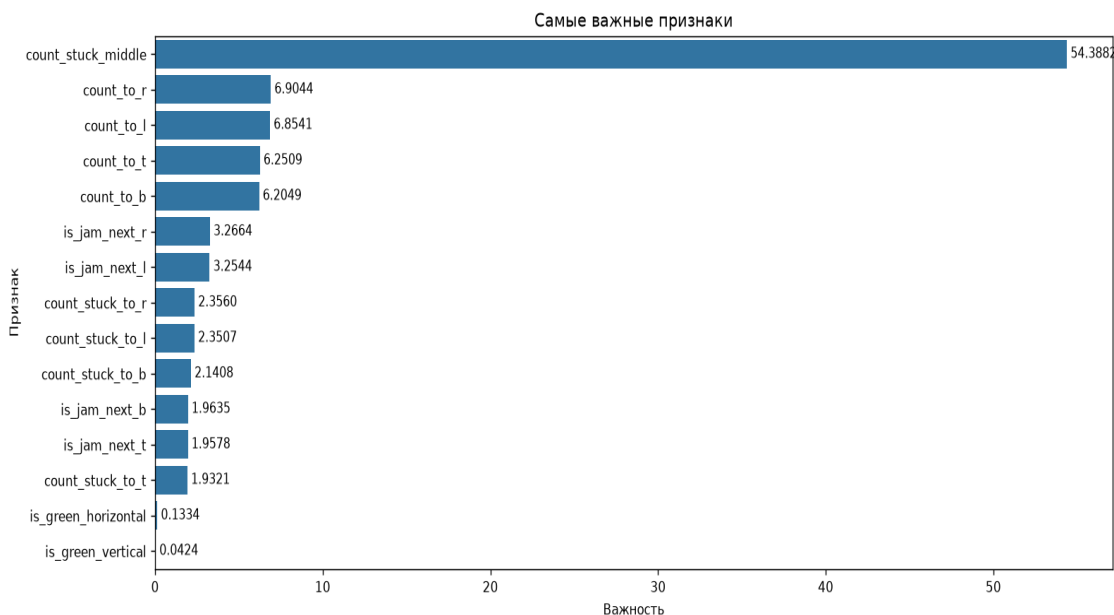


Рис 29. Полезность признаков для предсказания.

Использование модели было имплементировано в класс сегментации видеоряда. На рисунке 30 модель предложила пустить все машины в горизонтальном направлении.

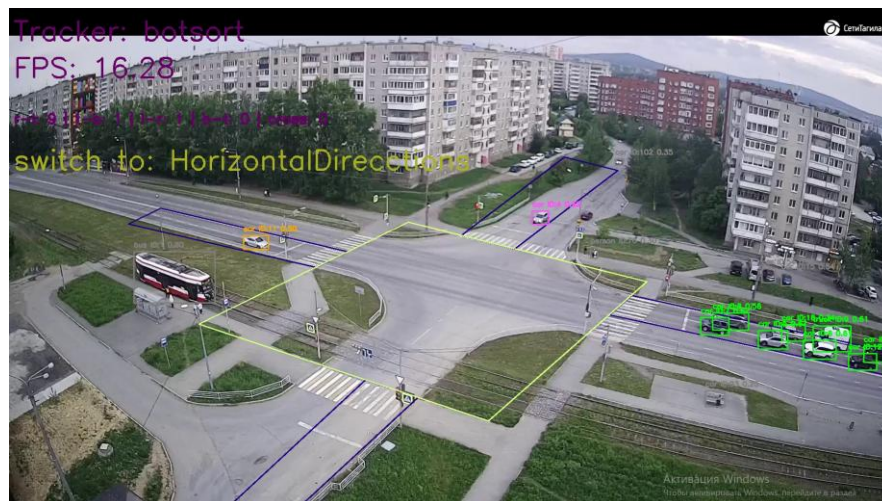


Рис 30. Предсказание горизонтального движения.

На рисунке 31 модель предложила пустить все машины в вертикальном направлении, из-за того, что в класс добавлен псевдо-параметр, переключающий светофор каждые 10 секунд и на момент кадра, машины в направлении сверху вниз простояли 2 фазы светофора в одном направлении.



Рис 31. Предсказание вертикального движения.

На рисунке 32 модель предложила включить красный сигнал для всех направлений

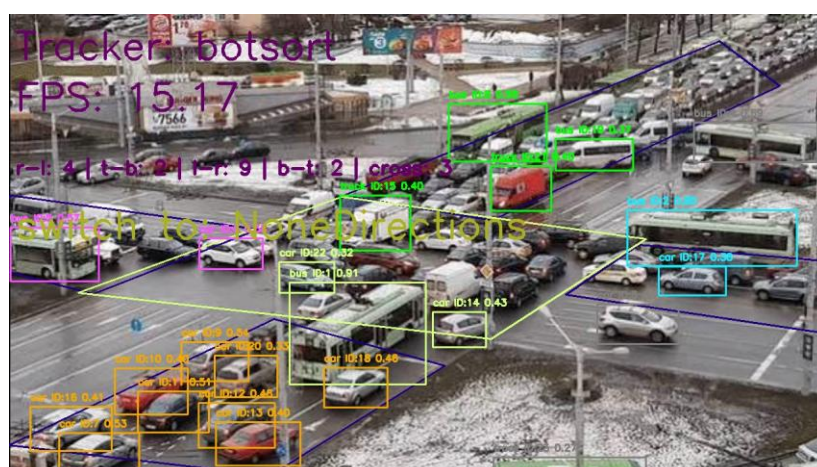


Рис 32. Предсказание «Красной фазы» для всех направлений.

5. СИМУЛЯЦИЯ ДОРОЖНОГО ДВИЖЕНИЯ С ИСПОЛЬЗОВАНИЕМ SUMO

SUMO — это открытый, высокопортативный пакет симуляции трафика, предназначенный для работы с крупными сетями. Он позволяет проводить симуляцию, объединяющую несколько видов транспорта (легковой, грузовой и т.д.), включая пешеходов, и оснащается большим набором инструментов для создания сценариев [23]. Для модификации объектов дорожной сети или получения информации о них используется библиотека *traci*. *Traci* (Traffic Control Interface) – это API-модуль, предоставляющий возможность взаимодействия с симулятором движения SUMO в режиме реального времени [24].

Будут рассмотрены:

- методы инициализации дорожной сетки и трафика машин,
- методы для регулирования дорожного движения,
- методы для анализа дорожного движения,
- результаты симуляции дорожного движения.

В качестве результатов будут представлены результаты метрик, собранных на симуляциях с различными параметрами и их сравнение.

5.1. Методы инициализации дорожной сетки и трафика машин

Для быстрой инициализации сети дорог был разработан метод *initialize*, в метод передается размерность сетки, ширина полос, и длина дорог. На рисунке 33 представлена выполняемая с помощью *os.system* команда. В консоли системы вызывается метод *netgenerate* со следующими параметрами:

- *--grid* (создать сетку)
- *--grid.number=* (размер сетки)
- *-L=* (количество полос на каждой из дорог)
- *--tls.guess* (задать шаблон для пересечений дорог)
- *--grid.length* (длина каждой из дорог сетки)
- *--output-file=* (выходной файл сети, записанный в xml формат)

```
def initialize(grid=3,lanes=2,length=200):  
    os.system("netgenerate --grid --grid.number="+str(grid)+" -L="+str(lanes)+" --tls.guess  
--grid.length="+str(length)+" --output-file=grid.net.xml")
```

Рис 33. Метод для инициализации дорожной сети.

Далее будет рассмотрен метод для задания остальных параметров симуляции *single*. В метод симуляции передается количество машин в симуляции (*vehicles*), путь до папки

SUMO на компьютере, в которой хранятся скрипты для быстрого развертывания сети (*path*), папка для записи свойств каждой из машины в симуляции, а также результатов самой симуляции (*gridDataFolder*), параметр *showVisualization* отвечает за включение визуализации симуляции во время прогона, *horizontal_light_duration* и *vertical_light_duration* – продолжительности зеленого сигнала для горизонтального и вертикального потоков соответственно, *withModel* – параметр задающий режим работы интеллектуальный или обычный. На рисунке 34 представлены команды для генерации потоков (используется скрипт из папки инструментов SUMO – *randomTrips.py*), транспортных маршрутов, вероятности поворотов на перекрестках (вероятность что автомобиль поедет в выбранном направлении, используется скрипт из папки инструментов SUMO – *generateContinuousRerouters.py*) соответственно.

```
def single(vehicles, path, gridsDataFolder, showVisualization = False,
horizontal_light_duration = 30, vertical_light_duration = 30, withModel = False
):
    sumoRunCfg = 'sumo-gui' if showVisualization else 'sumo'
    os.system('python "' + path + 'randomTrips.py" -n grid.net.xml -o flows.xml --begin 0 --end
1 --period 1 --flows '+str(vehicles))
    os.system("jtrrouter --flow-files=flows.xml --net-file=grid.net.xml --output-
file=grid.rou.xml --begin 0 --end 10000 --accept-all-destinations")
    os.system('python "' + path + 'generateContinuousRerouters.py" -n grid.net.xml --end 10000 -
o rerouter.add.xml')
```

Рис 34. Инициализация поведения на сетке дорог.

Для инициализации самих светофоров и детекторов, для подсчета автомобилей используется метод *create_additional_with_e1_detectors*, который принимает на вход путь до файла с сетью дорог «*grid.net.xml*» и путь куда будет сохраняться информация во время симуляции для каждого из детектора. Суть метода заключается в том, что перед каждой дорогой на пересечении (*junction*) устанавливаются датчики, считывающие движение, информация по каждой секунде итерации записывается в файл *detector_{id_перекрестка}_{номер_полосы}.xml*. На рисунке 35 представлена часть алгоритма, отвечающая за описанное выше поведение.


```

# Находим все junction и их входящие lane
for junction in root.findall("junction"):
    junction_id = junction.get("id")
    if not junction_id:
        continue

    # если не светофор
    if junction.get("type") != "traffic_light":
        continue

    junction_sumolib = None
    # Получаем объект перекрестка из sumolib
    try:
        junction_sumolib = net.getNode(junction_id)
    except:
        print(f"Предупреждение: перекресток {junction_id} не найден в sumolib")
        continue

    # Ищем все connection, ведущие к этому junction
    connections = root.findall(f".//connection[@tl='{junction_id}']")
    lanes = set()

    for conn in connections:
        edge_id = conn.get("from")
        lane_idx = conn.get("fromLane")
        if edge_id and lane_idx:
            lanes.add(f"{edge_id}_{lane_idx}")

    # Создаем детектор E1 и E2 (LaneArea) для каждой полосы
    for i, lane_id in enumerate(lanes):
        lane_id_direction = f"{lane_id}_{getLaneDirection(lane_id)}_{i}"
        # print(lane_id_direction)
        # ===== Детектор E1 (inductionLoop) =====
        detector = ET.SubElement(additional_root, "inductionLoop")
        detector.set("id", f"{junction_id}_E1_{lane_id_direction}")
        detector.set("lane", lane_id)
        detector.set("pos", "-10")
        detector.set("period", "1")
        os.makedirs(path_to_save_data, exist_ok=True)
        detector.set("file", f"{path_to_save_data}/detector_{junction_id}_{i}.xml")

```

Рис 35. Инициализация детекторов для каждой полосы перед светофором.

Для выделения областей перед светофором, играющих роль зоны наблюдения, в этой же функции с помощью API-метода `ET.SubElement(additional_root, "laneAreaDetector")` добавляются области *laneAreaDetector*, продолжительностью 150 метров перед светофором для каждой из полосы перед светофором и продолжительностью 10 метров для каждой из полосы внутри перекрестка. Для каждой из зон приписывается постфикс, с ее направлением движения. Размеченные зоны пригодятся для получения информации от ситуации на перекрестке в текущий момент.

На рисунке 36 представлен результат полной инициализации сетки.

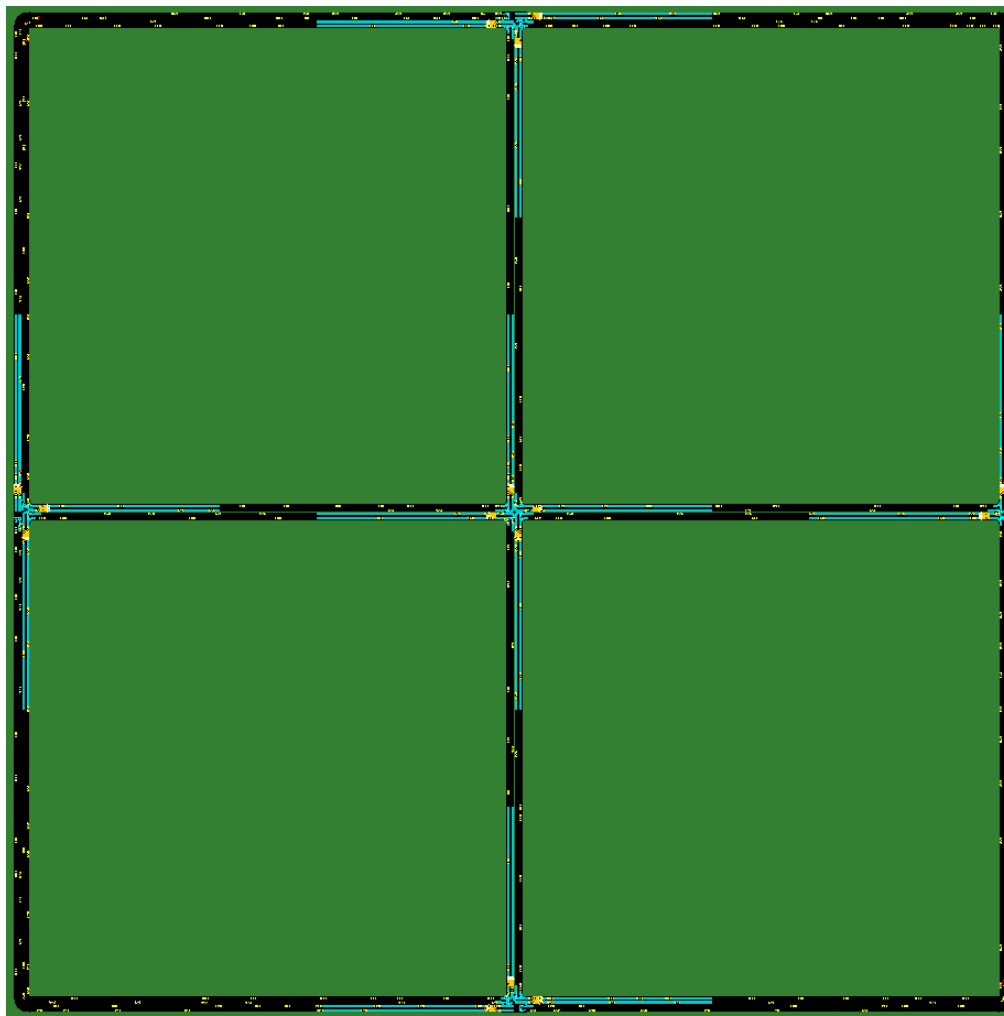


Рис 36. Инициализированная сетка 3x3 с рабочим светофором (длительность сигнала 20 сек) и 200 машинами в симуляции (синим цветом обозначены зоны наблюдения за трафиком).

5.2. Методы для регулирования дорожного движения

Для установки продолжительности сигналов светофора был написан метод *setTrafficLightFazes*, представленный на рисунке 37. Сначала извлекаются все полосы по часовой стрелке, начиная с правой стороны для которых функционирует светофор. Далее инициализируются 4 фазы, которые будут гореть для каждого из направлений, первое направление – горизонтальное, между основными сигналами добавляются переходные, с моргающим желтым сигналом. Данная логика с 4-я фазами записывается в каждый светофор с использованием API-метода *traci.trafficlight.setCompleteRedYellowGreenDefinition(trafficId, logic)* для цикличной работы светофора.

```

def setTrafficLightFazes(id, horizontal_light_duration, vertical_light_duration, yellow_light_duration = 3.0):
    # Все светофоры идущие по часовой стрелке с правой lane
    n_signals = traci.trafficlight.getControlledLanes(id)
    print(n_signals)
    phases = []
    # Установка продолжительности зеленого сигнала для горизонтального направления
    # Разбиваем на 4 списка по условиям:
    listRight = [x for x in n_signals if 'RIGHT' in getLaneDirection(x)] # Все lanes с направлением вправо
    listLeft = [x for x in n_signals if 'LEFT' in getLaneDirection(x)] # Все lanes с направлением влево
    listUp = [x for x in n_signals if 'UP' in getLaneDirection(x)] # Все lanes с направлением вверх
    listDown = [x for x in n_signals if 'DOWN' in getLaneDirection(x)] # Все lanes с направлением вниз

    stateHorizontalGreen = 'g'*len(listLeft) + 'r'*len(listUp) + 'g'*len(listRight) + 'r'*len(listDown)
    stateToVerticalGreen = 'y'*len(listLeft) + 'r'*len(listUp) + 'y'*len(listRight) + 'r'*len(listDown)
    stateVerticalGreen = 'r'*len(listLeft) + 'g'*len(listUp) + 'r'*len(listRight) + 'g'*len(listDown)
    stateToHorizontalGreen = 'r'*len(listLeft) + 'y'*len(listUp) + 'r'*len(listRight) + 'y'*len(listDown)

    phases.append(traci.trafficlight.Phase(horizontal_light_duration, stateHorizontalGreen, 0, 0))
    # Установка продолжительности переключения на красный сигнал для горизонтального направления
    phases.append(traci.trafficlight.Phase(yellow_light_duration, stateToVerticalGreen, 0, 0))
    # Установка продолжительности красного сигнала для горизонтального направления
    phases.append(traci.trafficlight.Phase(vertical_light_duration, stateVerticalGreen, 0, 0))
    # Установка продолжительности переключения на зеленый сигнал для горизонтального направления
    phases.append(traci.trafficlight.Phase(yellow_light_duration, stateToHorizontalGreen, 0, 0))
    logic = traci.trafficlight.Logic("0", 0, 0, phases)

```

Рис 37. Метод для установки продолжительности сигналов светофора.

Если Симуляция запускается с использованием интеллектуального переключателя светофора для основных фаз задается минимальная продолжительность в 10 секунд и с помощью метода *trafficLightsFazesForPredict* для основной логики программы добавляется ещё две фазы:

- промежуточная фаза, запускающая мигание желтого сигнала для всех светофоров,
- фаза, устанавливающая всем направлениям красный цвет.

После добавления новых фаз в логику, «Красная фаза» устанавливается для всех направлений. Метод *trafficLightsFazesForPredict* представлен на рисунке 38.

```

def trafficLightsFazesForPredict(tls_id):
    '''Добавляет в программу "Все красные", чтобы установить всем направлениям красный цвет'''
    program = traci.trafficlight.getCompleteRedYellowGreenDefinition(tls_id)
    phases = []
    phases = list(program[0].phases)
    n_signals = len(traci.trafficlight.getControlledLanes(tls_id))

    # Фаза перед включением всех красных сигналов
    before_red_phase = traci.trafficlight.Phase(4, 'y' * n_signals, 0, 0)
    # Фаза всех красных сигналов
    all_red_phase = traci.trafficlight.Phase(1, 'r' * n_signals, 0, 0)

    # Добавляем фазу в программу
    phases.append(all_red_phase)
    phases.append(before_red_phase)
    # print(phases)
    # Устанавливаем новую программу
    logic = traci.trafficlight.Logic("0", 0, 0, phases)
    traci.trafficlight.setProgramLogic(tls_id, logic)
    traci.trafficlight.setPhase(tls_id, len(phases) - 2)

```

Рис 38. Метод для добавления дополнительных фаз для интеллектуального управления.

Управление сигналами светофора в интеллектуальном режиме осуществляется с использованием функции *update_signal*. Алгоритм функции представлен на рисунке в приложении Б. Управление светофором с использованием модели заключается в переключении добавленных фаз с использованием API-функции *traci.trafficlight.setPhase(tls_id, phaseId)*.

5.3. Методы для анализа дорожного движения

Функция *updateInfoMap* используется для обновления информации для всей системы светофоров. Алгоритм сбора информации дорожного движения работает аналогично алгоритму, представленному в пункте 3 на рисунке 22, только вместо использования геометрии объекта для определения принадлежности объекта к одной из зон, к информации об объектах в каждой из зон можно обратиться с помощью запроса *traci.lanearea.getLastStepVehicleIDs(line_id)*. Помимо этого, теперь для светофора известно о наличии пробок на соседних светофорах. Вся собранная ситуация в текущий момент записывается в pandas dataframe и используется для предсказания следующей полезной фазы с помощью функции *addInfoToTLDataInfo*, представленной на рисунке 39.

```
def addInfoToTLDataInfo(currentInfoMap, tlsId):
    currentTLSInfo = currentInfoMap[tlsId]
    info_collected = {
        'count_to_r': len(currentTLSInfo['objects_stuck_to_r']),
        'count_to_l': len(currentTLSInfo['objects_stuck_to_l']),
        'count_to_b': len(currentTLSInfo['objects_stuck_to_b']),
        'count_to_t': len(currentTLSInfo['objects_stuck_to_t']),
        'count_stuck_middle': getTrullyStuck(_counter=currentTLSInfo['objects_stuck_middle']),
        'count_stuck_to_r': getTrullyStuck(_counter=currentTLSInfo['objects_stuck_to_r']),
        'count_stuck_to_l': getTrullyStuck(_counter=currentTLSInfo['objects_stuck_to_l']),
        'count_stuck_to_b': getTrullyStuck(_counter=currentTLSInfo['objects_stuck_to_b']),
        'count_stuck_to_t': getTrullyStuck(_counter=currentTLSInfo['objects_stuck_to_t']),
        'is_jam_next_l': currentTLSInfo['is_jam_next_l'],
        'is_jam_next_r': currentTLSInfo['is_jam_next_r'],
        'is_jam_next_t': currentTLSInfo['is_jam_next_t'],
        'is_jam_next_b': currentTLSInfo['is_jam_next_b'],
        'is_green_horizontal': currentTLSInfo['is_green_horizontal'],
        'is_green_vertical': currentTLSInfo['is_green_vertical'],
    }
    dfInfo = pd.DataFrame(info_collected, index=[0])
    dataframe = pd.concat([currentTLSInfo['trafficInfoDF'], dfInfo], ignore_index=True)
    return info_collected, dataframe, dfInfo
```

Рис 39. Функция для записи данных о ситуации в области светофора в текущий момент.

Кроме информации, необходимой для предсказания также для каждого светофора собираются данные о количестве пробок, количеству времени, проведенном на красном сигнале светофора, количеству машин, участвовавших в пробке за все время симуляции. Эти данные пригодятся для анализа и определения эффективности предложенного в работе метода управления светофором.

5.4. Результаты симуляции дорожного движения

Для сравнения с результатами симуляции движения с использованием интеллектуальных светофоров, также было запущено несколько симуляций с обычным светофором, со следующими параметрами:

- количество машин [200, 400, 600, 800, 1000],
- время работы зеленого сигнала светофора [20, 30, 42, 50, 60],
- длина полотна, между светофорами 500 м,
- продолжительность симуляции 10000 сек (2,8 часов).

Для анализа эффективности интеллектуального светофора были предложены следующие метрики:

- jams_count – количество пробок на светофоре за все время симуляции,
- jams_car_count – общее количество машин, побывавших в пробке на светофоре за все время симуляции,
- total_cars_passed – общее количество машин, которые проехали через светофор,
- real_time_on_red - общее время, проведенное на красном сигнале светофора за все время симуляции.

Далее будут рассмотрены результаты для светофора В1, стоящего на перекрестке, светофоре А1, стоящем на развилке и общие результаты для всех светофоров total.

На рисунке 40 представлено количество машин, проехавших через светофор для разных режимов в симуляции с разным количеством автомобилей.

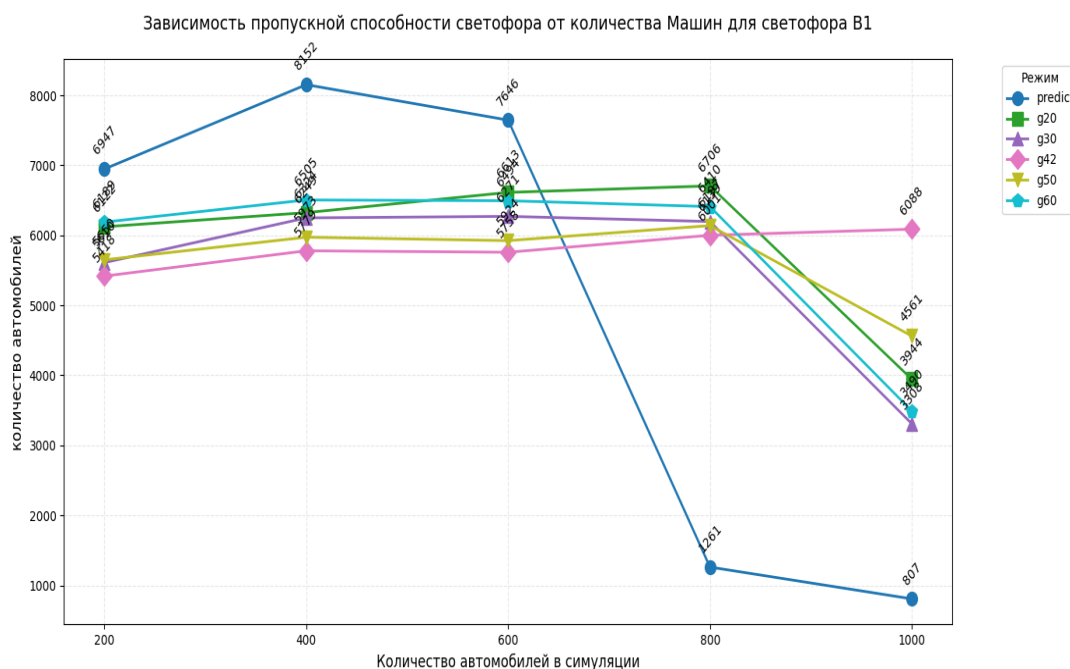


Рис 40. Зависимость пропускной способности светофора от количества машин для светофора В1.

По результирующему графику видно, что интеллектуальный светофор показывал лучшие результаты на симуляциях с 200 – 600 автомобилей (прирост 15-25%), но на симуляции с 800, и 1000 автомобилей эффективность пропускной способности резко упала. Если посмотреть на график, представленный на рисунке 41, заметно, что для предсказательной симуляции с увеличением машин в симуляции значительно возросло время пребывания на красном сигнале светофора.



Рис 41. Зависимость общего времени, проведенного на красном сигнале от количества машин в симуляции для светофора В1.

Если же посмотреть на метрики количества машин, участвовавших в пробках и количества пробок, представленных на рисунках 42 и 43, можно заметить, что количество пробок приблизилось к 0 не только для предсказательной, но и для симуляций с продолжительностью времени зеленого сигнала 42, 50, 60.

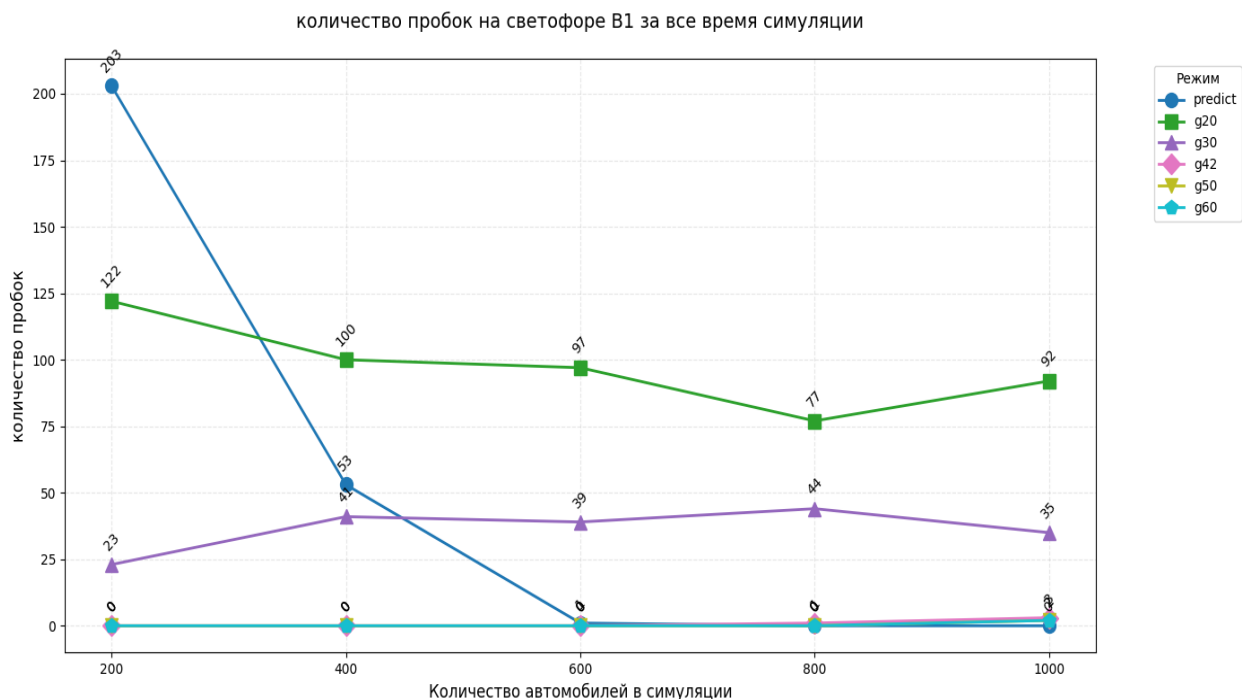


Рис 42. Количество пробок на светофоре В1 за все время симуляции.

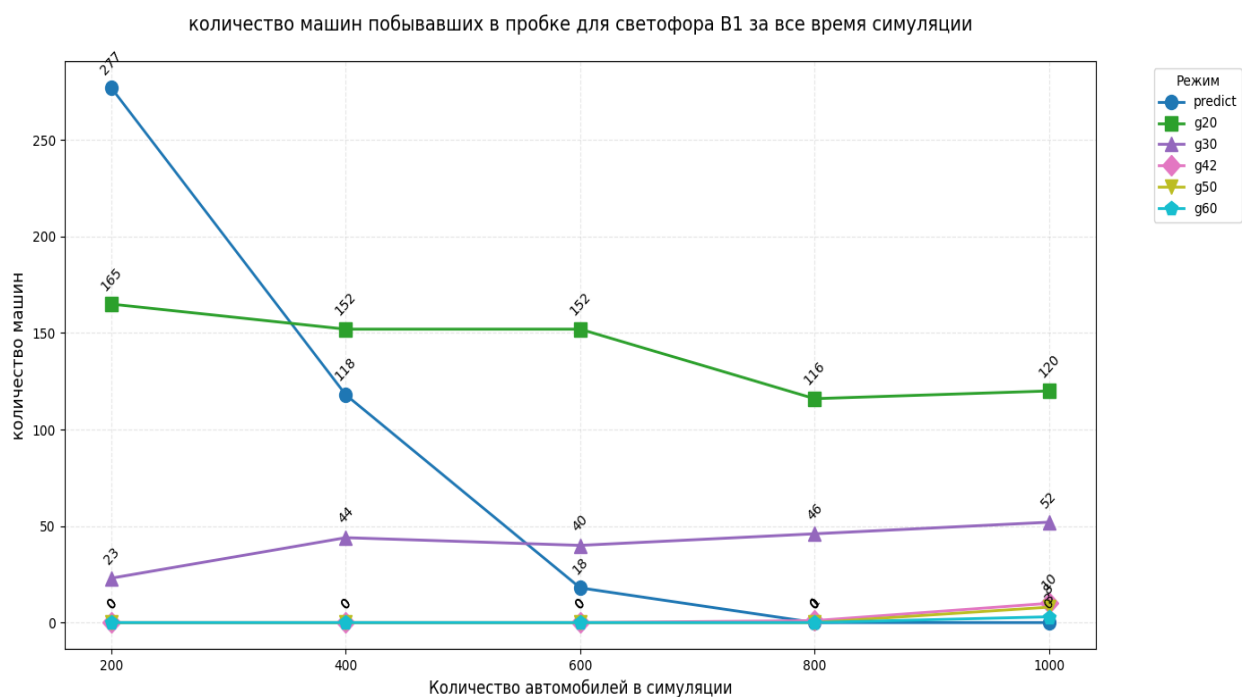


Рис 43. Количество машин, побывавших в пробке для светофора В1 за все время симуляции.

Если рассматривать светофор, который управляет тремя дорогами ситуация похожая, для количества пропущенных машин (Рис 44), но для количества общего времени, проведенного машинами в пробке, метрика стала расти не только для предсказательной ситуации (Рис 45), то же самое можно сказать про оставшиеся две метрики (Рис 46 – 47).

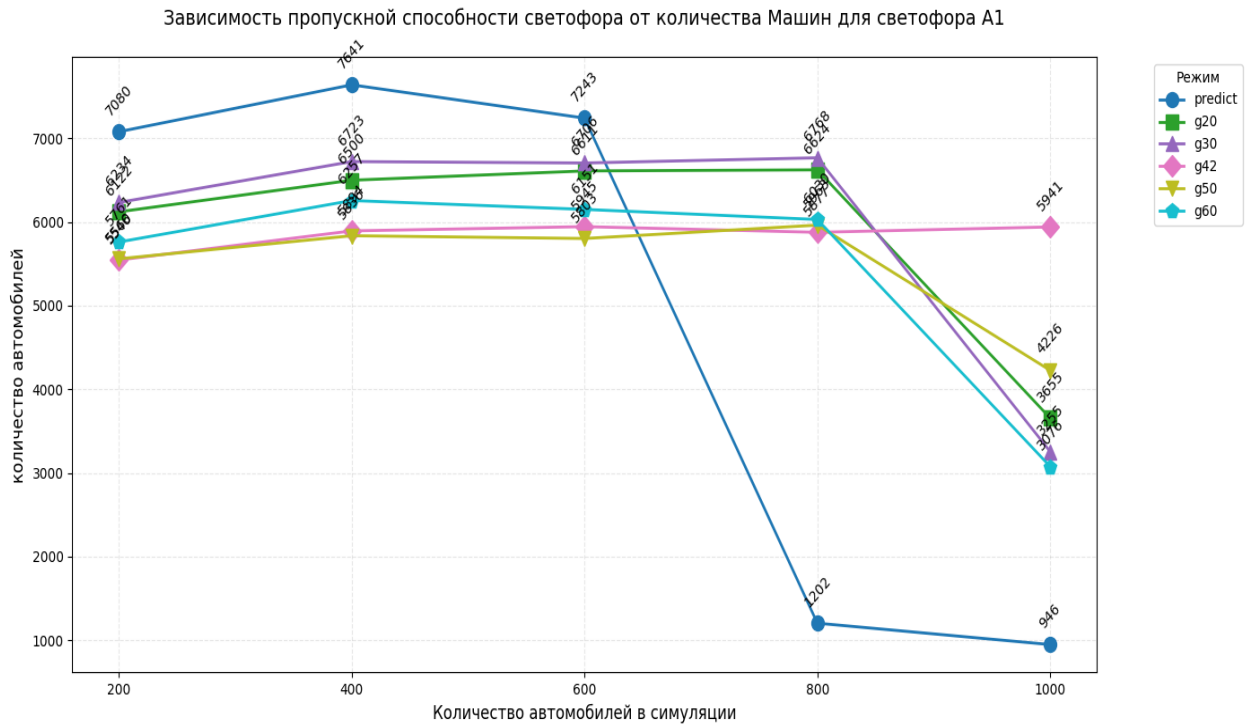


Рис 44. Зависимость пропускной способности светофора от количества машин для светофора А1.

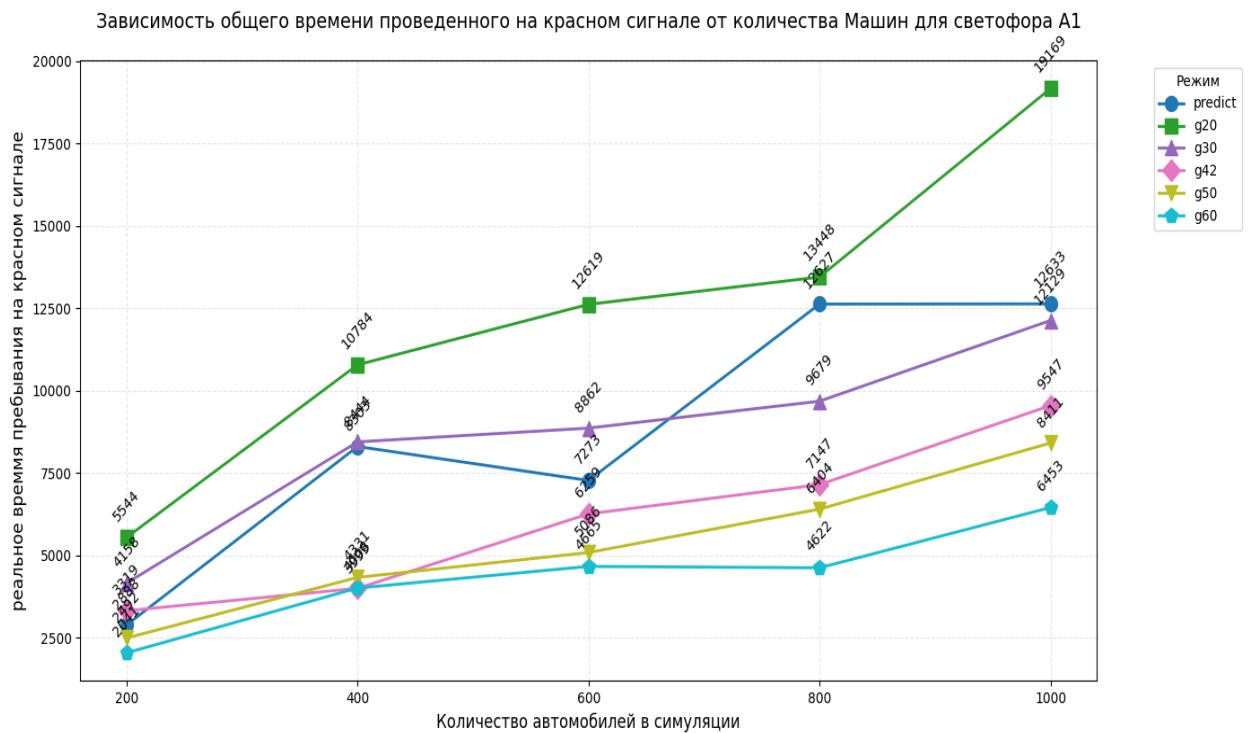


Рис 45. Зависимость общего времени, проведенного на красном сигнале от количества машин в симуляции для светофора А1.

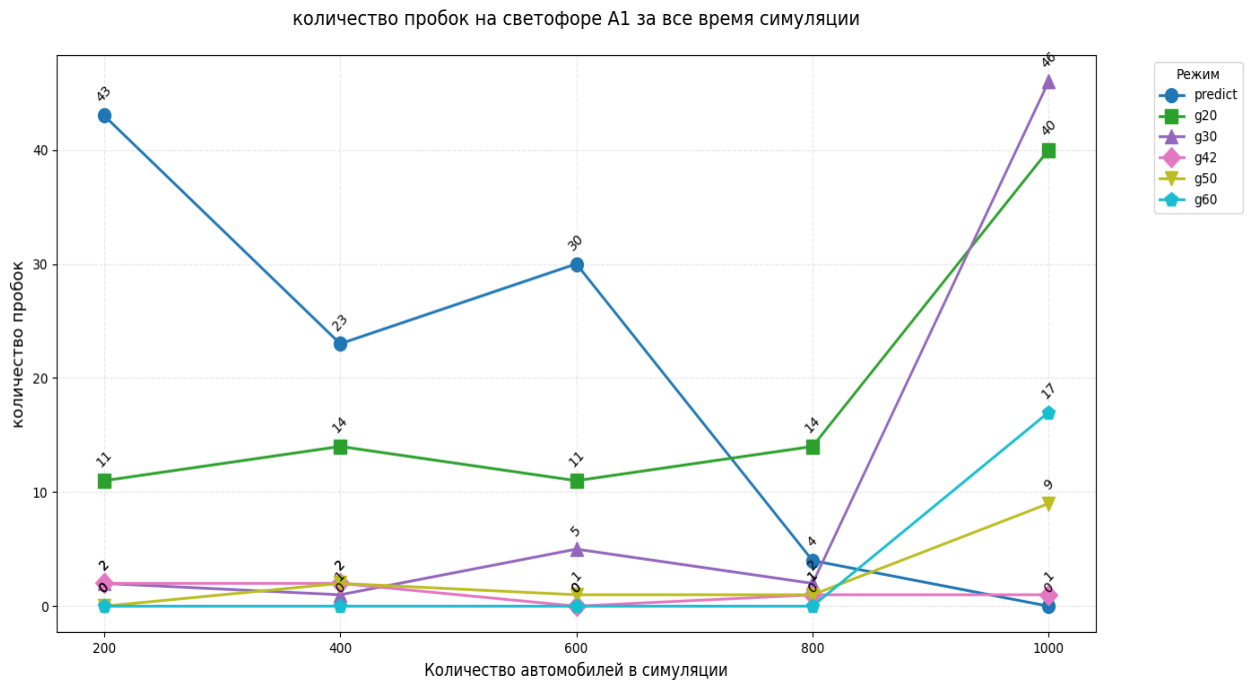


Рис 46. Количество пробок на светофоре А1 за все время симуляции.

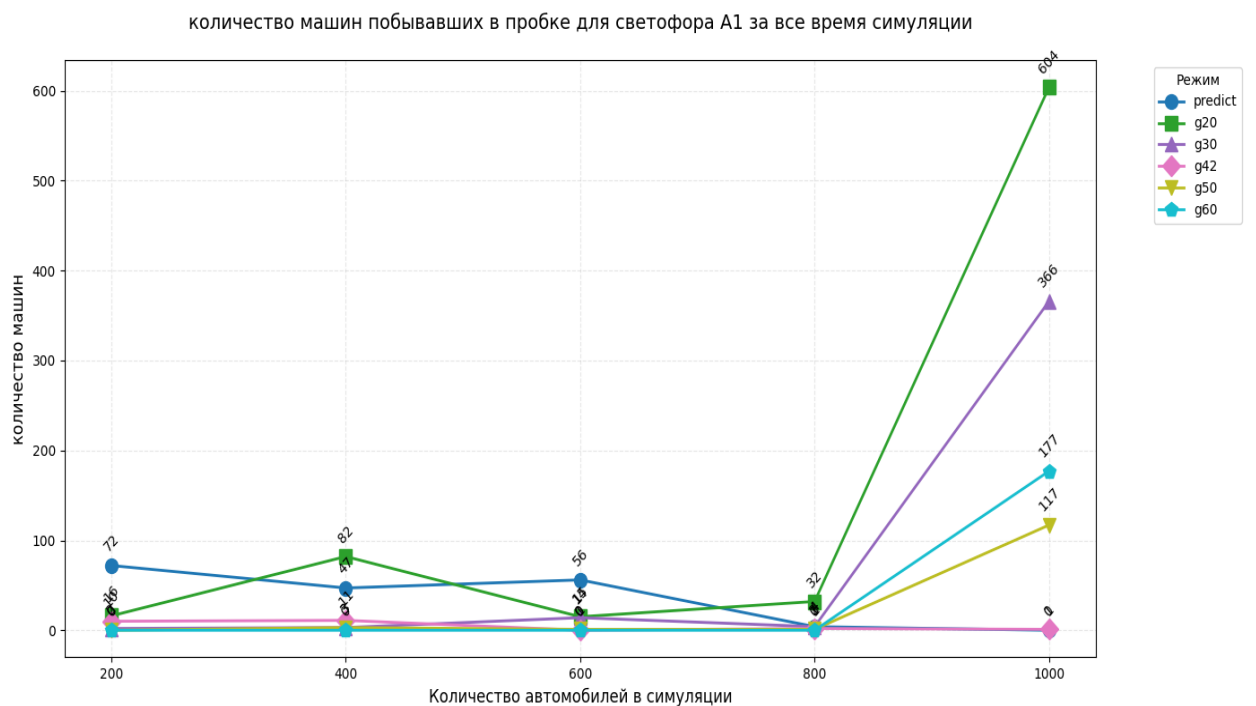


Рис 47. Количество машин, побывавших в пробке для светофора А1 за все время симуляции.

Для всех светофоров в системе метрики представлены на рисунках 48 и 49. На метриках также виден общий прирост пропускной способности светофора на 10-15% на симуляциях с 200 – 600 машинами и заметное увеличение времени, проведенного в пробках.

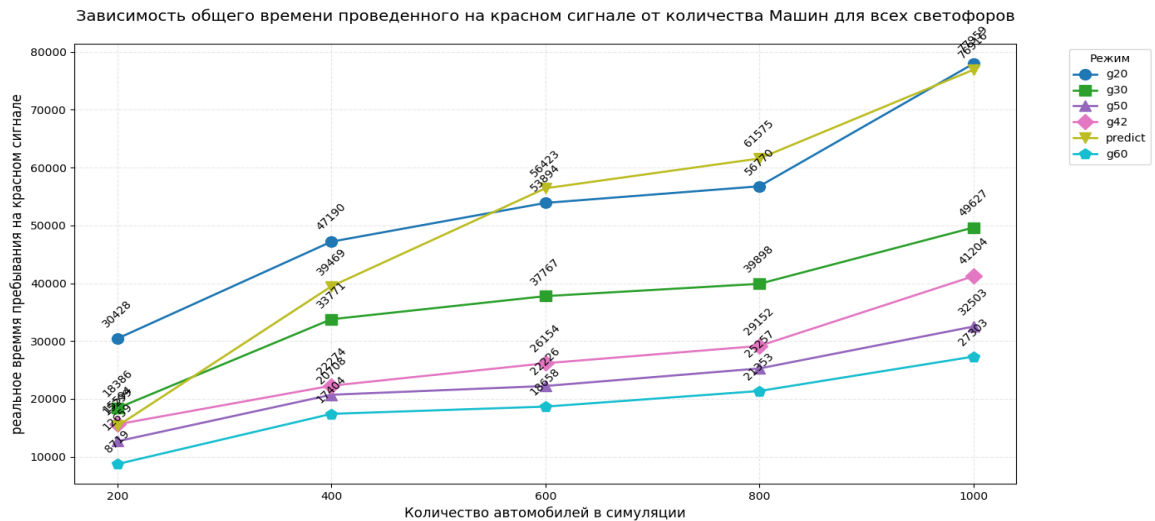


Рис 48. Зависимость общего времени, проведенного на красном сигнале от количества машин в симуляции для всех светофоров.

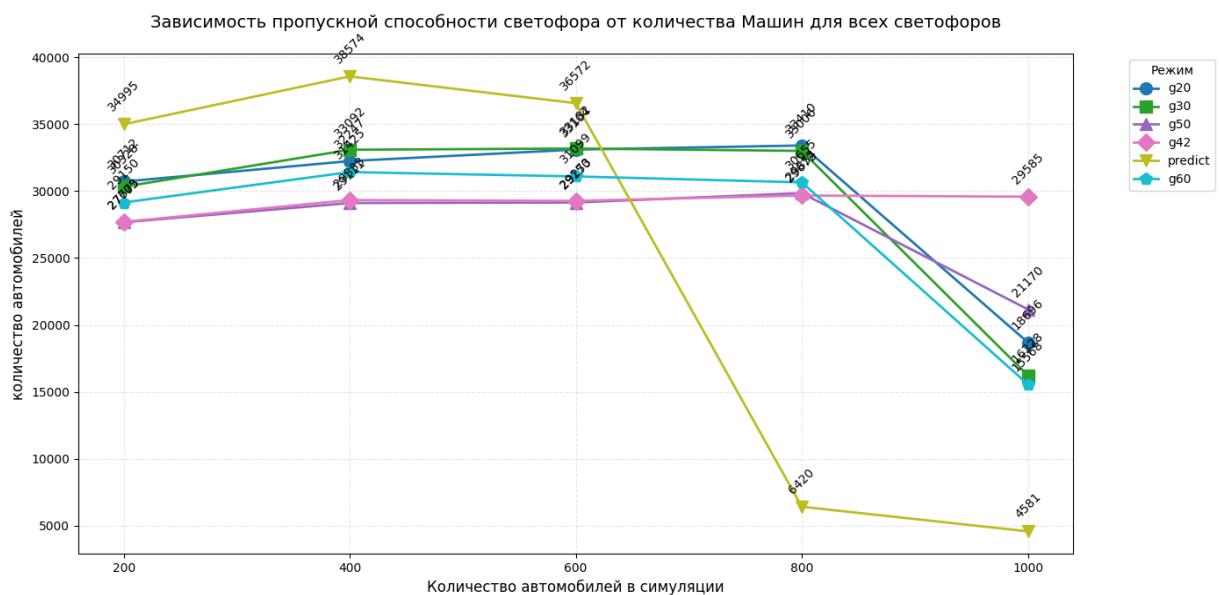


Рис 49. Зависимость пропускной способности светофора от количества машин для всех светофоров.

По результатам анализа симуляций модель предсказания сигнала светофора демонстрирует корректную работу на части сценариев, однако чувствительна к выбору параметров, описывающих дорожную ситуацию. Так, например, в данной работе критерий образования пробки основан преимущественно на переключениях фаз светофора, что приводит к ограниченному эффекту в ряде конфигураций трафика. Это указывает на необходимость более информативных показателей перегруженности, таких как, скорости потока и времени ожидания на красном сигнале, и их интеграции в модель управления.

ЗАКЛЮЧЕНИЕ

В результате выполнения выпускной квалификационной работы был изучен материал, необходимый для построения архитектуры модели и рассмотрены проекты, решающие похожие задачи. Был разработан класс, использующий модель YOLO26l-seg, для сегментирования видео с целью определения каждого пикселя на принадлежность к транспортным средствам или людям и получения по собранным данным информации о ситуации на перекрестке. Разработанный класс был протестирован на подготовленных заранее записях дорожного движения. Обучена модель для регулирования дорожного движения. Также, была развернута система симуляции дорожного движения SUMO и проведен анализ полученных после множества симуляций данных.

По результатам симуляций в системе SUMO для ряда сценариев удалось получить увеличение общей пропускной способности потока на 10–15% по сравнению с базовым режимом работы светофоров. Но в части экспериментов улучшение оказалось ограниченным, что связано с упрощённым критерием выявления пробок в текущей версии алгоритма.

Были выполнены задачи, а именно:

- проведен анализ существующих решений адаптивного управления светофором,
- выбрана Baseline архитектура и представлена ее модифицированная версия,
- разработать программу для анализа дорожной ситуации с использованием компьютерного зрения,
- обучена классификационная модель на данных, сформированных после работы программы анализа дорожного движения и представлены результаты сегментации видеопотока с использованием этой модели,
- развернута система контроля трафика и реализован алгоритм управления сигналами светофора на основе предсказаний модели,
- выполнен запуск симуляций с разными параметрами для обычного светофора и для светофора, управляемого нейросетевой моделью,
- проведены анализы симуляций.

После выполнения всех задач реализован полный конвейер обработки данных: от получения видеок кадров и сегментации объектов с помощью YOLO до преобразования результатов в признаки дорожной ситуации, принятия решения моделью управления и проверки эффективности в среде симуляции SUMO.

Таким образом, цель и поставленные задачи были достигнуты, а также показано, что даже при простых правилах управления на основе CV можно добиться заметного прироста пропускной способности для части сценариев.

Разработанную систему можно усовершенствовать следующим образом, доработать метод определения пробок, добавить, как один из параметров, наличие пешеходов на переходах, дообучить модель на специфичных данных. После успешных прогонов улучшенной модели в системе SUMO, провести полевые испытания.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1 А.Б. Рахманов. Четыре колеса апокалипсиса: причины автомобильных пробок в крупных странах мира [Научная статья] // Вестник Томского государственного университета Философия. Социология. Политология. 2020. № 58 – С. 1. - URL: <https://cyberleninka.ru/article/n/chetyre-kolesa-apokalipsisa-prichiny-avtomobilnyh-probok-v-krupnyh-gorodah-mira/viewer> (Дата последнего обращения: 20.06.2026)
- 2 В США подсчитали, во сколько обходятся стране автомобильные пробки [Электронный ресурс] // URL: <https://officelife.media/news/54320-v-ssha-podschitali-vo-skolko-obkhodyatsya-strane-avtomobilnye-probki-summa-vpechatlyayet/> (Дата последнего обращения: 20.06.2026)
- 3 Ученые Пермского Политеха разработали алгоритм оптимизации перекрестков на основе компьютерного моделирования [Электронный ресурс] // URL: <https://pstu.ru/news/2025/10/17/17823/> (Дата последнего обращения: 20.06.2026)
- 4 Как сделать светофор умнее, а проезжать и загруженные перекрестки быстрее? [Электронный ресурс] // URL: <https://habr.com/ru/articles/553162/> (Дата последнего обращения: 20.06.2026) .
- 5 На «умные» светофоры в Уфе в 2023 году потратили более 270 млн рублей [Электронный ресурс] // URL: <https://www.kommersant.ru/doc/6509735> (Дата последнего обращения: 20.06.2026).
- 6 Видеодетектор автотранспортных средств ИНФОПРО [Электронный ресурс] // URL: <https://www.infoprocess.ru/products.html> (Дата последнего обращения: 20.06.2026).
- 7 Видеодетектор автотранспортных средств ИНФОПРО сценарии использования [Документация] <https://www.infoprocess.ru/downloads/scenario.pdf> (Дата последнего обращения: 20.06.2026).
- 8 НПО ИТС: Паук Трафик [Электронный ресурс] // URL: <https://clck.ru/3QxVQh> (Дата последнего обращения: 20.06.2026).
- 9 В Германии тестируют умные светофоры Ими управляют самообучающиеся алгоритмы [Электронный ресурс] // URL: <https://digitalocean.ru/n/v-germanii-testiruyut-umnye-svetofory> (Дата последнего обращения: 20.06.2026).
- 10 Светофоры с искусственным интеллектом начали испытывать в Германии [электронный ресурс] // URL: <https://kolesa.kz/content/news/svetofory-s-iskusstvennym-intellektom-nachali-ispytyvat-v-germanii/> (Дата последнего обращения: 20.06.2026).
- 11 KI4LSA Künstliche Intelligenz für Lichtsignalanlagen. (KI4LSA Искусственный интеллект для Системы световой сигнализации) [Научная статья] // URL:

- https://www.iosb-ina.fraunhofer.de/content/dam/iosb/iosb-ina/documents/WhitePaper_KI4LSA.pdf (Дата последнего обращения: 20.06.2026).
- 12 ANAKHI HAZARIKA, MOUSTAFA M. NASRALLA, NIKUMANI CHOUDHUR, SOHAIB BIN ALTAF KHATTAK, IKRAM UR REHMAN. Edge ML Technique for Smart Traffic Management in Intelligent Transportation Systems [Научная статья] // Digital Object Identifier 10.1109/ACCESS.2024.3365930 – С. 25447 - URL: <https://drive.google.com/file/d/1zYmKNS5f-iOmcibpucmLvaoQdne4uqS0/view?usp=sharing> (Дата последнего обращения: 20.06.2026).
 - 13 Ultralytics YOLO26 – Обзор [Документация] // URL: <https://clck.ru/3UGcEc> (Дата последнего обращения: 20.06.2026).
 - 14 Семантическая сегментация: самый полный гайд 2024. [Электронный ресурс] // URL: https://habr.com/ru/companies/data_light/articles/855336/ (Дата последнего обращения: 20.06.2026).
 - 15 Nikhileswara Rao Sulake Guide on YOLOv11 Model Building from Scratch using PyTorch. [Документация] // URL: <https://www.analyticsvidhya.com/blog/2025/01/yolov11-model-building/> (Дата последнего обращения: 20.06.2026).
 - 16 CatBoost: алгоритм градиентного бустинга [Электронный ресурс] // URL: <https://practicum.yandex.ru/blog/algorithm-gradientnogo-bustinga-catboost/> (Дата последнего обращения: 20.06.2026).
 - 17 Instance Segmentation [Документация] // URL: <https://docs.ultralytics.com/tasks/segment/> (Дата последнего обращения: 20.06.2026).
 - 18 Как получить запись с городских камер видеонаблюдения [Электронный ресурс] // URL: <https://clck.ru/3QxVyt> (Дата последнего обращения: 20.06.2026).
 - 19 Документация Ultralytics Roboflow [Документация] // URL: <https://docs.ultralytics.com/ru/integrations/roboflow> (Дата последнего обращения: 20.06.2026).
 - 20 Open Source Computer Vision [Электронный ресурс] // URL: <https://docs.opencv.org/3.4/index.html> (Дата последнего обращения: 20.06.2026) или <https://opencv.org/> (Дата последнего обращения: 20.06.2026).
 - 21 Документация Ultralytics Roboflow - трекинг [Документация] // URL: <https://docs.ultralytics.com/ru/modes/track> (Дата последнего обращения: 20.06.2026).
 - 22 Применение алгоритмов компьютерного зрения для системы контроля отгрузки горячекатанных рулонов [Выпускная квалификационная работа] // 2024 г. – С. 21-24

- URL: https://elar.urfu.ru/bitstream/10995/140349/1/m_th_a.e.vasiljev_2024.pdf (Дата последнего обращения: 20.06.2026).
- 23 SUMO User Documentation [Документация] // URL: <https://sumo.dlr.de/docs/index.html> (Дата последнего обращения: 20.06.2026).
- 24 TraCI [Документация] // URL: <https://sumo.dlr.de/docs/TraCI/index.html> (Дата последнего обращения: 20.06.2026).

Ссылка на репозиторий с кодом программы и отчетами

Репозиторий проекта, вместе с отчетами можно найти, перейдя по ссылке:

https://github.com/Uranus28/gitVKR_trafficLightCV_SUMO (github.com)

Алгоритм переключения сигналов светофора в SUMO

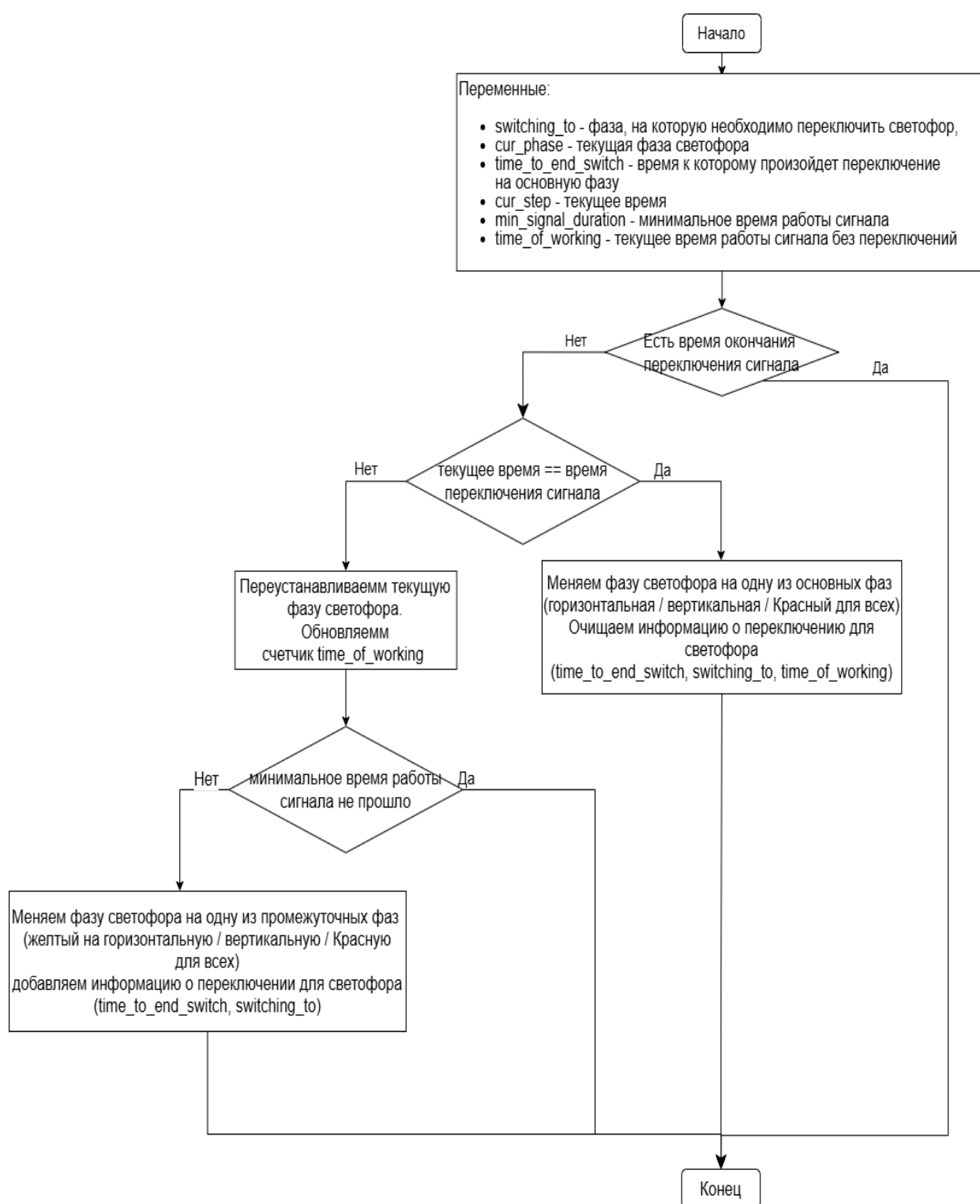


Рис 1. Алгоритм переключения сигналов светофора в SUMO.